

University of Padova

Department of Mathematics "Tullio Levi-Civita"

Master Thesis in Data Science

Explainable Predictive Monitoring for Object-Centric Processes

Supervisor
Professor Massimiliano de Leoni
University of Padova

Co-supervisor
Doctor Alessandro Padella
University of Padova

Master Candidate
Otto Guillermo Wantland Conde

Student ID
2185663

Academic Year
2025-2026

"Pessimismo dell'intelligenza, ottimismo della volontà"
— Antonio Gramsci

Abstract

Object-centric processes is a paradigm which represents process executions as sequences of events wherein each event may be related to different objects of multiple types, such as orders or items. Predictive monitoring on object-centric processes seeks to provide stakeholders with actionable predictions on currently active processes by analyzing the information found in object-centric event logs. The need for more accurate predictions on a wider variety of applications has led these methodologies to adopt deep learning methods, such as neural networks, for their analysis. However, these black box methods are known for their lack of transparency, which can make their adoption in critical business areas more difficult as stakeholders are hesitant to trust models they do not properly understand.

In order to address this issue this thesis proposes a four-component explainability framework for use with predictive process monitoring on object-centric event logs. The approach combines both factual and counterfactual explanation methods to provide stakeholders with a holistic understanding of a given model's predictions.

The framework was tested on four different regression models, trained on information from two different ~~OCEL 2.0 datasets~~. The result is the identification of the most impactful elements of an input on a given model's prediction and an estimation of the impact those elements have on the model's output. Additionally, through a counterfactual ~~explanation~~ the framework is able to identify similar ~~subgraphs within the universe of~~ process executions that can help the stakeholder understand the minimum number of changes ~~to a process~~ that may lead to a different prediction. These results are presented to stakeholders in the form of graphs and tables, which aim to make the information as approachable as possible to the end user.

Contents

Abstract v

List of figures ix

List of tables xi

Listing of acronyms xiii

1 Introduction 1

2 Related Works 5
- 2.1 Process Mining . . . 5
  - 2.1.1 Object-centric processes vs. traditional processes . . . 6
- 2.2 Object-centric predictive process monitoring . . . 6
  - 2.2.1 From tabular to object-centric encodings . . . 7
- 2.3 Graph neural networks (GNNs) . . . 7
  - 2.3.1 A direct comparison baseline for heterogeneous GNNs . . . 8
- 2.4 Explainable AI . . . 9
  - 2.4.1 Explainability on Neural Networks . . . 9
  - 2.4.2 Explainability on predictive process monitoring . . . 10
  - 2.4.3 The GNN-explainability landscape . . . 10
- 2.5 Summary and proposed framework . . . 16

3 Preliminaries 17
- 3.1 Single-id event logs and predictive process monitoring . . . 17
- 3.2 Object-centric event logs . . . 18
- 3.3 Predictive analytics on object-centric processes . . . 22
- 3.4 Explanation on process predictions . . . 23
  - 3.4.1 Perturbation-based analysis . . . 24
  - 3.4.2 Gradient/feature-based analysis . . . 26
  - 3.4.3 Counterfactual explanations . . . 27
  - 3.4.4 Explanation-quality metrics . . . 28

4 Models 31
- 4.1 Datasets . . . 31
  - 4.1.1 Order Management Dataset . . . 31
  - 4.1.2 Logistics Dataset . . . 32
  - 4.1.3 Dataset Processing . . . 33
- 4.2 Heterogeneous Graph Neural Network for Predictive Process Monitoring . . . 34
  - 4.2.1 Train/test split of the datasets . . . 34
  - 4.2.2 Model architecture choice and evaluation metrics . . . 35
  - 4.2.3 Hyperparameter sweep . . . 36

4.2.4 Training and validation . . . 36
4.2.5 Baselines for comparison . . . 37
4.3 Explainability Framework Design . . . 37
4.3.1 Perturbation-based explanation (LOO + Shapley Values) . . . 37
4.3.2 Gradient/feature-based explanation (Gradient · Input and Integrated Gradients) . . . 38
4.3.3 Counterfactual explanation . . . 38
4.3.4 Fidelity metrics . . . 39

5 Results 41
5.1 GNN experiments . . . 41
5.1.1 Hyperparameter sweep . . . 41
5.1.2 Model comparison versus homogeneous GNN . . . 43
5.1.3 Model comparison against baselines . . . 46
5.2 Explainability framework . . . 49
5.2.1 Perturbation-based explanation . . . 49
5.2.2 Gradient/feature-based explanation . . . 53
5.2.3 Counterfactual explanation . . . 55
5.2.4 Explanation metrics . . . 57

6 Conclusion 67

References 69

Acknowledgments 73

Listing of figures

5.1 Normalized Val MAE scores for the HTG model per hyperparameter setting. Note: scales (y-axis) are not aligned, as I intended to compare hyperparameter settings within each KPI. . . 42
5.2 Normalized Val MAE scores for the HGT model on the Logistics dataset for KPI "Elapsed time from CustomerOrder to Depart" per different number of layers 43
5.3 Normalized Val MAE scores for the HGT model on the Logistics dataset for KPI "Elapsed time from CustomerOrder to LoadToVehicle" per different number of layers 44
5.4 Training and validation loss learning curves of the trained HGT model and the homogeneous GNN on the best performing model of the Order Management dataset. 45
5.5 Training and validation loss learning curves of the trained HGT model and the homogeneous GNN on the best performing model of the Logistics dataset. 46
5.6 Performance of the chosen HGT model and three baseline models. Note: the best-performing hyperparameter configurations are used for HGT 47
5.7 Test MAE for each regression model across prefix depth. 48
5.8 Fidelity metrics of LOO method against GNNExplainer for KPIs of each dataset 49
5.9 Measure of agreement between LOO method and full Shapley Values 50
5.10 Graphs showing the single-trace explanations generated indicating the most important nodes in each trace as well as the impact on the model's prediction. 51
5.11 Graphs showing the single-trace explanations generated indicating the most important edges in each trace as well as the impact on the model's prediction. 52
5.12 Explanation subgraphs created using the identified most important nodes and edges, one worked example per KPI. 53
5.13 Graphs showing the single-trace explanations of node importance generated for an early prefix and the last-event prefix of the same process execution, for each of the four KPIs. 54
5.14 Graphs showing the aggregate explanations generated indicating the most important nodes for a collection of 50 traces as well as the impact on the model's prediction. 55
5.15 Global LOO+Shapley explanation, ranked across all identified elements, for Order Management / Orders → PayOrder (50 traces). 56
5.16 Global LOO+Shapley explanation, ranked across all identified elements, for Order Management / Orders → PackageDelivered (50 traces). 57
5.17 Global LOO+Shapley explanation, ranked across all identified elements, for Logistics / CustomerOrder → Depart (50 traces). 58
5.18 Global LOO+Shapley explanation, ranked across all identified elements, for Logistics / CustomerOrder → LoadToVehicle (50 traces). 59
5.19 Graphs showing the single-trace explanation provided for feature importance on the Order node using the Gradient · Input method for identification and the LOO-Shapley method for quantifying the feature's impact on the prediction. 59
5.20 Graphs showing the single-trace explanation provided for feature importance on the Order node using the Integrated Gradients method for identification and the LOO-Shapley method for quantifying the feature's impact on the prediction. 60

5.21 Graphs showing the aggregate explanation provided for feature importance of the nodes in the chosen traces using the Gradients · Input method for identification and the LOO-Shapley method for quantifying the feature's impact on the prediction. 60
5.22 Graphs showing the aggregate explanation provided for feature importance of the nodes in the chosen traces using the Integrated Gradients method for identification and the LOO-Shapley method for quantifying the feature's impact on the prediction. 60
5.23 Graph representation of the user's selected graph for counterfactual explanation. 61
5.24 Graph representation of the counterfactual graph identified for counterfactual explanation. . . 62
5.25 Graph representation of the dissimilarity values for the distinct elements compared between the user's chosen graph and the chosen counterfactual. 63
5.26 Table showing the other possible counterfactual graph candidates found in the candidate set with their prediction value and dissimilarity scores 63
5.27 Table showing the differences in the features of the Orders node of the two subgraphs. The total delta of the values is shown in the final column. 64
5.28 Table showing the differences in the features of the Event nodes of the two subgraphs. The number of events of each type is shown for the designated graph along with an estimation of their impact on the model's prediction. 64
5.29 Graphs showing the baseline (full graph), fidelity+ (explanation removed), and fidelity- (explanation only) subgraphs used to compute the fidelity metrics meant for the stakeholder. 64
5.30 Table of the explanation metrics obtained. 65

Listing of tables

2.1 Comparison of the GNN-explainability methods surveyed in this section. 15

3.1 Object-Centric Event Log With Associated Objects 19

4.1 Summary of the datasets used in this thesis. 32
4.2 Final chosen hyperparameters for this thesis's two datasets. Logistics is split into its two KPIs since, unlike Order Management, they do not share the same final configuration. 36

Listing of acronyms

PPM . . . . . Predictive Process Monitoring

OCEL . . . . . Object-Centric Event Log

GNN . . . . . Graph Neural Network

XAI . . . . . Explainable Artificial Intelligence

KPI . . . . . Key Performance Indicator

1 Introduction

Predictive process monitoring is a research discipline within the field of process mining that aims to provide stakeholders with a prediction on the outcome of an ongoing process [1]. These predictions are generated through the analysis of data organized into event logs, which represent process executions as a timestamped sequence of events. Traditionally, these logs would represent the event data as inputs where each event is associated with exactly one object, or process instantiation. While this representation was useful for modelling certain processes (such as insurance claims) which typically rely on a single object type being tied to each execution flow, other types of processes could not properly be described through this methodology without a significant loss of information. Object-centric processes, on the other hand, follow a different paradigm: rather than being executed in isolation, a process instance interacts with other instances of the same or different processes [2]. This paradigm aligns more closely with real-world applications, where a process unfolds through the parallel execution of different execution flows with different associated objects that interact with one another. Contemporary predictive process monitoring research has begun focusing on performing their predictions by using Object-centric Event Logs (OCELs). OCELs represent process flows through the relationship between events and the multiple objects associated with each event step, as opposed to single-case-ID logs that force an artificial single-object view [2].

The complex intricacy of the process instances that relate to each other via many-to-many associations poses a significant challenge for predictive analytics. Since traditional process mining models assume that events are associated with exactly one object, these techniques cannot be directly applied to object-centric event data. Modern applications have leveraged the power of deep learning models, such as neural networks, to address this situation. While traditional machine learning models such as Gradient Boosted Trees (GBT) or neural network architectures such as Long Short-Term Memory (LSTM) can successfully be applied to OCELs they require the object-centric event data to be encoded in a tabular format which may lead to the loss or misrepresentation of information. Graph Neural Networks (GNNs) are specialized frameworks that extend traditional neural network methodologies so they may be used on graph-based datasets. By extending traditional neural networks to a graph-based domain, these models are able to provide predictions based not only on the input's features but also on the un-

derlying structure of the information, such as the ways events and objects interact throughout the execution of a process.

Recent research has shown success in the application of graph-based encodings that respect the OCEL's underlying structure [3] and allow for GNNs to be used for predictive analysis. While these graph-based encodings originally used homogeneous graph structures, more recent works have explored whether heterogeneous structures can provide better predictive results. However, the evidence so far is inconclusive, with results showing improvement on some datasets but tying or underperforming on others [4] [5]. Similarly, while graph neural networks have proven to be extremely effective at providing accurate predictions on a variety of datasets their use in predictive process monitoring tasks has led to conflicting results, with some works citing a noticeable improvement [3] and others citing worse performance than traditional neural network models [6]. Additionally, graph neural networks (like their non-graph counterparts) offer limited transparency regarding the way they compute their predictions. These difficulties can hinder adoption in a business-process setting where a wrong or unexplained prediction has real operational consequences.

The need for providing end users with comprehensive explanations on the predictions made by deep learning models has led to the rise of Explainable Artificial Intelligence (XAI). XAI methodologies rely on the application of methods such as Shapley Value analysis to provide insight into the importance a model places on the different elements present in the input data. These methods have been successfully applied in other fields, such as Natural Language Processing, and have also found adoption in traditional (non-object-centric) processes. However, ~~my~~ research shows a gap in the application of explainability methods to predictive process monitoring on object-centric processes. In order to address this issue I propose an explainability framework composed of four complementary elements, each covering a distinct question a stakeholder might ask (which elements mattered, which raw features drove them, what minimal change would change the prediction, and how much to trust the explanation). The first is a perturbation-based explanation which uses a combination of a Leave-One-Out methodology [7] to identify important elements and a Shapley Value analysis [8] to properly quantify their impact on a model's prediction, providing a clear look at what elements mattered in the input. Second, a feature/gradient-based explanation that complements the first method with a finer-grained view of which raw features drive the important elements previously highlighted [9]. Third, a counterfactual explanation which demonstrates what changes to the factual input would be needed to achieve a different output prediction [10], complementing the previous factual explanations with a more actionable look at what input changes can lead to a different prediction. And, finally, a set of fidelity metrics to assess the validity of the provided explanations [11], complementing the past three layers by giving the stakeholder a measure of how much they can trust the given explanations. The goal of this framework is not only to provide explanations on the chosen models, but also to present this information in a comprehensive manner to process stakeholders who may have no formal understanding of predictive methods as a whole. A more detailed discussion on each chosen component and their importance for the framework is provided in the state-of-the-art review in Chapter 2.

To validate the proposed framework, my thesis presents a set of experiments conducted on two synthetic OCEL 2.0 datasets. The experiments seek to train four distinct predictive models, two for each dataset, aimed at predicting the time remaining until the occurrence of a given event in the process. With the models properly trained and validated the explainability framework is applied to all four in order to identify the key elements driving each prediction, as well as quantify their impact on the prediction. These values are then presented in the form of graphs and tables in order to provide a stakeholder with an easy to interpret output that would help them better

understand the predictive model.

Overall, the results show that my proposed framework is able to identify important elements in the model and show it in a way that can generate trust in the stakeholder. A clear limitation of the framework is the associated computational cost of quantifying each element's impact on the prediction, but it is expected that further research on explainability frameworks for heterogeneous graphs will address this limitation.

The remainder of the document is organized as follows. Chapter 2 provides a review of related works on predictive process monitoring on object-centric event logs as well as explainability methods. Chapter 3 introduces the theoretical background ~~on the topics introduced in Chapter 2~~. Chapter 4 provides a rundown of the work done to train the predictive models that were used to test the proposed explainability framework. Chapter 5 shows the results of the processes outlined in Chapter 4. Chapter 6 summarizes the findings and limitations of the proposed framework.

2
Related Works

In this chapter, I provide an overview of the related work on predictive process monitoring, from its role as a process mining methodology to its early approaches using flattened Object-Centric Event Logs (OCELs), followed by contemporary applications using heterogeneous graph structures, where I identify a key research gap, the absence of explainability analysis on heterogeneous predictive process monitoring for OCELs. Recognizing this gap, I examine methodologies on explainable artificial intelligence (XAI) used in other domains to provide an enhanced understanding of complex predictive models. Building on these insights I propose a four-component explainability framework to enhance existing predictive process monitoring pipelines by providing stakeholders with a transparent overview of the model's functionality.

2.1 Process Mining

As data availability has increased exponentially so too has the need to use this data to provide actionable insights. Process Mining is a research discipline that seeks to act as a bridge between the fields of data mining and business process modeling and analysis. Its goal is to discover, monitor, and improve real processes by analyzing the events that make up a process. These events are presented in the form of an event log, a sequential record of events, such that every event refers to an activity (a well-defined step in the process) and is related to a particular case or process execution [1].

Aside from the sequence of events, event logs may also store additional information about each step in the process such as the person who performs the process, the timestamp of the event or a set of features related to elements tied to the event (such as the size of an order).

Process mining can be broken down into three main types: process discovery, conformance checking, and process enhancement. Process discovery is the most prominent of the process mining techniques and refers to a methodology that transforms an event log into a model without any a priori information. This technique is crucial

as it can show business stakeholders that the information present in an event log related to a process execution that may be further analyzed. Conformance checking, on the other hand, is a technique that validates whether a given model properly reflects the reality shown in the provided event log and vice versa. It is a diagnostics process meant to ensure the proper functioning of models created during process discovery.

Finally, process enhancement techniques seek to extend or improve an existing process model using the information present in the event log. Their main goal is to take an encoding of features of the event data as input and deliver insights, predictions, or actions as output [3]. These insights can help identify problems in the underlying process, such as bottlenecks, or quantify important elements such as throughput times. Predictive process monitoring (PPM) methods stem from process enhancement techniques, their main goal being to provide a process's stakeholder with a prediction on the output of a currently running process execution. Through this prediction the stakeholder may be able to identify problems in the current execution and take preventive action in order to ensure the proper resolution of the process execution.

2.1.1 Object-centric processes vs. traditional processes

Traditional process mining techniques operate under two assumptions: Each object is of the same type and each event is associated with exactly one object. While these assumptions may be valid in certain cases, such as the handling of insurance claims (where each object describes an instantiation of an insurance claim) [3], they do not properly represent most real-world processes. In real-life organizations, processes do not often happen in isolation, instead, several instances of different processes that are related by a set of objects can be executed simultaneously and interact with one another. In order to address this discrepancy a new paradigm was developed: object-centric processes. In object-centric processes an event may be related to objects of different types and an event with multiple objects may have multiple predecessor events [12]. These object-centric processes are represented through Object-Centric Event Logs (OCELs). OCELs store information on multiple aspects of a process through the use of object types. For example, an OCEL representing an Order Management process will contain instances of different events in the process, such as the placing of the order or the creation of a package containing the ordered products, as well as information on the distinct objects that are related to each distinct event, such as the items ordered or the vehicle assigned for the package delivery. The resulting log's structure more closely resembles a graph rather than the sequential structure that was assumed for use in traditional process mining.

Acronym already Defined

2.2 Object-centric predictive process monitoring

Predictive process monitoring (PPM) on object-centric event logs (OCELs) refers to a process that monitors the running instances of a given object-centric process and provides predictions on key performance indicators (KPIs). The goal of these predictions is to alert stakeholders of possible risks arising in the process so they may mitigate their effects.

2.2.1 From tabular to object-centric encodings

Process mining techniques, such as process enhancement, seek to provide insights by analyzing the information provided by event logs. Early process enhancement techniques encode features of event data in a tabular form where each row contains the feature values for an event. However, since each process execution is a temporally ordered sequence of events, summarizing the event data in a tabular form removes this temporal structure. Sequential encodings were then developed as a way to address this problem. By representing the data as a sequence of feature values the temporal structure of the information could be preserved. This, in turn, allowed for the use of specialized models that could leverage this sequential structure, such as LSTMs, to provide more accurate KPI predictions.

While these methods are effective in analyzing and providing predictions on traditional event logs, object-centric processes and their OCELs provide a significant challenge due to their added complexity. In order to apply traditional process enhancement techniques on OCELs initial implementations flattened the OCELs and encoded the object interactions as tabular features. This initial approach was successful in showing the benefit the additional complexity found in OCELs could provide to predictive models. When combined with state of the art machine learning models such as Gradient Boosting predictions performed on object-centric processes showed better results than those trained using a naive approach [2]. However, while this method provides results the flattening process has major consequences on the quality of the calculated features. Flattening the OCEL can lead to events disappearing, a problem called deficiency. Conversely, events can also appear more than once, causing a problem called convergence. Finally, flattening can also cause the creation of misleading directly-follows relations, a problem labeled divergence. For example, an OCEL graph may show two events happening simultaneously but, when placed in a tabular form, the events are forced into an ordered state which creates a false directly-follows relation between one event and another. To combat this loss in feature quality Adams et al. proposed a graph-based encoding that retains the structural information of the process by maintaining the original graph structure of the OCEL [3]. The encoding also proposes an object-centric adaptation of features calculated for traditional event data [13]. The need for this translation arises from the two main differences between traditional event data and object-centric data: each event can have multiple predecessors/successors and each event might have multiple related objects of different types. All the feature calculations are adapted to the graph structure and are grouped according to different perspectives: Control-Flow, Data-Flow, Resource, Performance and Objects.

2.3 Graph neural networks (GNNs)

The graph-based encoding proposed by Adams et al. can then be used to train Graph Neural Networks (GNNs) to provide a prediction on the chosen KPIs. GNNs are special implementations of neural networks that are specifically created to handle graph-based inputs and outputs. They're designed to leverage not just the features present in each node, but also the inherent relationship between objects in a graph. GNNs learn via a message-passing mechanism, built from two functions: an aggregation function and an update function [14]. In order to learn about the input structure the GNN model collects messages from nearby nodes to a particular node, which are then aggregated based on information gathered from each node's own neighborhood. This information is used to create node embeddings. The process continues recursively, with each iteration providing nodes with more

information from their surrounding neighbors. Each iteration also provides the embeddings with information from distant parts of the graph. In the first iteration each node holds information from its immediate neighbors, nodes that are one hop away. In the next iteration, nodes aggregate information from nodes two hops away and update each embedding. This process continues until each node's embedding includes data from its n-hop neighborhood. This architecture enables GNNs to capture and learn from the complex relational structures present in graph data, making them effective for various tasks involving graph-structured data, such as graph classification, node classification, and link prediction [14]. Predictive process monitoring follows a classic supervised learning structure, where the GNN is provided with a tuple of features (in this case a graph representation of a current process) and an output value Y (the KPI value for the given process). Because of their ability to leverage the structural information present in the graph encoding of the OCEL GNNs have shown improvement in their predictive qualities when compared with a flattened encoding such as a tabular OCEL. Subsequent research has shown that by retaining the OCEL's structural information this graph-based encoding is able to outperform state of the art approaches using a flattened dataset [15]. However, it's worth noting that a different work found the opposite to be true. A direct comparison between LSTM/Gradient Boosting models and GNN techniques across two different processes and 21 KPIs found that GNNs underperformed, while also taking considerably more time to train [6].

2.3.1 A direct comparison baseline for heterogeneous GNNs

All the graph-based encodings mentioned so far worked with homogeneous graphs. These types of graphs are constructed under the assumption that all nodes represent the same object type. In the case of OCEL encoding, this approach requires aggregating object features in order to present the graph information as a series of interconnected event nodes. While graph encoding was able to properly address the problems caused by flattening the OCEL it still lacked the ability to properly encode all the distinct relationship types that existed in the event log between objects of different types and their associated events. In order to address this loss of information a heterogeneous graph structure was proposed that would treat each object instance as its own unique node type. Smit et al. [4] propose a framework for heterogeneous object-event graph encoding (HOEG) defined by two architectural choices: the use of a heterogeneous encoding that does not aggregate object features and the use of a k-dimensional heterogeneous GNN for its predictions. Under this encoding, it is no longer necessary to aggregate the distinct object features into each event node, instead, each object node holds its own set of distinct features directly related to that object. For example, a createOrder event no longer needs to aggregate the object information for the order and any other related object such as the items that make up the order. In a heterogeneous encoding each of those object types is given its own separate node with its own distinct features. Smit et al.'s work showed that expanding the representative power of the graph led to inconclusive results in the predictive quality of their models. While the heterogeneous structure provided better predictions on one of the tested datasets, it tied on a different one, and provided lower results on another. My thesis adopts their work as its direct comparison baseline in the Models chapter; however, it is necessary to address some key differences. First, Smit et al.'s k-dimensional GNN is a homogeneous GraphConv-based operator, made heterogeneous through PyTorch Geometric's generic to_hetero() transform. Second, their graph structures have no object-to-object edges, presenting only the event-to-event and object-to-event relations. Finally, HOEG predicts remaining time at every event node of a complete process execution in a single forward pass (no masking to one target), instead of once per prefix. For my work I chose to

utilize a heterogeneous model architecture based around HGT convolutions which are natively type-aware and require no model wrapper, the graph structure used to train and test this model contains all three types of node connections (object-to-event, event-to-event and object-to-object), and the training/testing sets are composed of distinct prefix subgraphs drawn from every process execution in the dataset.

2.4 Explainable AI

Explainable AI (XAI) refers to the field of study of methodologies used for providing end users with an understanding of the way complex machine learning models function.

2.4.1 Explainability on Neural Networks

Given the widespread availability of structured data more and more businesses are adopting predictive models for high-stakes decision-making processes. Simple predictive models such as decision trees are able to generate their own explanations and are referred to as white box methods. Their methodology is easily interpretable and an end user is able to understand why the model predicted a certain value by following the logic within the model. However, their simplicity also limits their overall predictive power and range of applications. Since the concrete goal of these models is to provide as accurate predictions as possible the current trend is to increase the predictive performance with the use of deep learning methods such as neural networks. While these methods offer a powerful and versatile solution to predictive problems their inherent complexity leads to a lack of transparency and an inability to interpret the given predictions. Identified as black box methods, these interpretability problems can lead to them not being used for high-stakes decision-making processes even with their superior performance.

The explainable artificial intelligence (XAI) paradigm seeks to address this issue by providing tools to help make these black box methods more understandable to the end user. XAI methods try to approximate the behavior of the model with post-hoc explainable techniques such as Shapley values, feature importance, attention weights, etc. While the concepts of explainability and interpretability are often used interchangeably a subtle distinction must be highlighted [11]. The first one refers to the ability to properly link the effect an input has on a model's output, i.e. how faithful the explanation is to the model. The second refers to the ability to express this relationship in an unambiguous and simple way so that an end user may properly use the information. A simple explanation may be easily interpretable but may not be faithful to the predictive model. Therefore, it is especially important to strike a balance between the two, aiming to provide explanations that are faithful to the model but also easily interpretable.

There exists a variety of XAI methodologies for providing interpretable explanations of a given model. Three widely used methods rely on perturbation (sometimes also referred to as ablation), feature attribution and counterfactual analysis. Perturbation methods seek to quantify the impact of removing or altering input values and measuring the effect this change has on the model's prediction. An example of these methods are Shapley Values. This explainability method adapts the game theory approach of Shapley Values, which seeks to find a fair distribution of a game's payout among the players that have collaborated. Under this method, the features of an instance correspond to the players and the payout is the difference between the prediction made by the predictive model and the average prediction. Thus, for each instance, the Shapley value of a feature provides a quantifiable

expression of its impact on the model's prediction [16]. Feature attribution methods seek to clarify the roles of input features in the prediction of the outcome and have been successfully used in different fields such as molecular science and Internet of Things projects. Counterfactual explanations seek to identify the minimal changes needed in an input in order to change the model's prediction. These methods have found success in domains such as recommendation systems and molecular science.

2.4.2 Explainability on predictive process monitoring

The proper application of explainability methods is especially important for predictive process monitoring since the adoption of black box models in critical business areas is tied to the trust stakeholders can place on the predictions being provided by the model. As Galanti et al. [17] show in their work on explainable support systems, process analysts and stakeholders will not adopt predictive models that do not put explanations as a core feature and build the user's trust. Because of this imperative, much work has been done to extend predictive process monitoring with explainability features. Mehdiyev et al. [18] provide a comprehensive survey of the existing work in this field. While their review shows a large amount of PPM applications using a variety of XAI methods the majority of the works reviewed adhere to the traditional process mining paradigm. The authors distinctly point out the need for further work in the field of object-centric process explainability, highlighting that the change in paradigm challenges traditional explanation approaches.

Further research into works focused on object-centric processes reveals the following. Weinzierl et al.[19], Wickramanayake et al.[20], and Pasquadibisceglie et al. [21] seek to explain the object-centric processes by focusing directly on the deep learning methods chosen. Their works provide explanations through the use of Layer-wise relevance propagation, attention mechanisms, and neuro fuzzy networks respectively. These methods show positive results but are limited by being model specific approaches that are specifically designed for certain model types. Model-agnostic approaches include Galanti et al. [17] [2] who adopted the use of Shapley Values as an explainability method for their object-centric process analysis, as well as, Hsieh et al.[22] and Huang et al.[10] who propose the use of counterfactual methods to provide explanations. While all these approaches, both model-centric and model-agnostic, successfully delivered interpretable explanations they were applied to tabular encodings of object-centric data and relied on traditional neural network implementations. On the other hand, works such as Adams et al.[3], Gherissi et al. [23], Smit et al.[4] and de Leoni & Volpato [5] focus on the use of graph-based encodings and GNNs for generating predictions. Of these, only Adams et al. offer a preliminary explainability component, applying SHAP to the individual edges of their homogeneous graph encoding; the remaining three do not provide any explainability method for interpreting their models' results, and none of the four extend explainability to a genuinely heterogeneous graph structure. As far as I could tell, no work has been done to apply explainability techniques on object-centric processes encoded as heterogeneous graphs.

2.4.3 The GNN-explainability landscape

Since current state of the art PPM methods are based around the use of GNNs it is essential to explore XAI methods specifically designed for graph-based models. Broad taxonomic surveys [24] have been conducted to map out the universe of options available in the space. Two main explanation categories exist: Factual and Counterfactual. Factual explanations aim to explain a model by identifying the input features that are the most impactful on the

model's prediction. Counterfactual explanations, on the other hand, seek to explain the model by identifying the minimum change to the input graph that would lead to a different prediction. Counterfactual explanations can be used to identify a set of related characteristics whose modifications can alter the model's prediction. Factual explanations can be further categorized as either post-hoc or self-interpretable. Self-interpretable techniques seek to incorporate interpretability directly into the architecture and training process of the GNN. Such explanations tend to employ graph attention techniques to assign attention weights to different nodes and edges, using these to identify the most prominent elements in the input graph structure. Post-hoc techniques, on the other hand, treat pre-trained GNNs as black boxes. These techniques can be broken down into instance-level approaches and model-level, according to their need to access the internal parameters of the model. Instance-level approaches include two further categorizations (among several) which were used in this thesis implementation: perturbation-based methods and gradients/feature-based methods.

Perturbation and gradient/feature-based methods

Perturbation-based methods seek to determine the characteristics or traits that are critical for the model to provide accurate predictions by manipulating the model's inputs and observing the changes caused in the output. Perturbation-based methods provide a straightforward way of understanding the function of elements in models.

Since GNNs are trained to obtain information from an input's nodes as well as its structure it is important that perturbations respect this structure as much as possible. In GNNs these perturbations are applied to the model in the form of masks, which limit the available information in the input that is passed to the predictive model without significantly altering the input's actual structure. Methods such as Shapley Values, mentioned above, or Leave-One-Out (LOO) methodologies [7], which apply the same logic as Shapley values but consider only one feature at a time as opposed to the entire feature space, can be extended for use with GNNs by applying this mask methodology. The main setback of Shapley Values is their computational intensity, having to obtain the exact coalition enumeration for a complex graph structure can be extremely costly. However, this cost can be mitigated through the use of estimation methods such as Monte Carlo permutation sampling described in Castro et al.'s ApproShapley algorithm [8][16].

Masks can also be directly optimized via backpropagation, allowing for methods that identify a mask which shows only the most relevant characteristics for an accurate prediction at a lower computational expense. The resulting masked version of the original graph constitutes a subgraph known as the explanation subgraph.

GNNExplainer is a perturbation method that utilizes masks to learn important patterns in both edge and node features. It is a model-agnostic implementation and identifies a succinct subgraph that is most relevant or influential for a GNN's prediction of a particular instance [25]. While the original GNNExplainer implementation was designed for homogeneous models it can be adapted for use with heterogeneous models by creating a separate mask per node type. However, a limiting factor of this implementation is that GNNExplainer is not able to perform edge-masking and cannot provide edge importance values for heterogeneous graphs.

PGExplainer is another perturbation based explanation method that utilizes masks to change the model's input graph. However, PGExplainer parametrizes mask generation across instances via a trained neural network instead of optimizing per-instance in order to improve generalization [26].

While both model options were successfully tested on the thesis dataset PGExplainer was decided against because its implementation required a greater degree of processing to properly integrate into the heterogeneous

framework. GNNExplainer is retained only as a standalone comparison point against the chosen implementation since its lack of edge importance severely limits its application and its assessment metrics were consistently lower than those of the chosen explanation methodology. Instead, the final perturbation-based explanation works on a combination of exhaustive LOO and Shapley Values. Exhaustive LOO analysis works as a cost efficient way to identify the most impactful elements in the input's structure by running on every feature of the input graph (nodes, edges and node features) and identifying those with the greatest impact on the prediction value. By limiting the explanation space to the elements identified through LOO and using Monte Carlo sampling I was able to use Shapley Values to properly quantify the impact those graph features have on the model's prediction without incurring major computational cost.

Gradients/feature-based methods identify key input features by calculating the gradients of the target prediction with respect to those features. These methods were not originally designed with GNNs in mind but, given their simplicity and generality, they were expanded into the graph domain [9]. These methods can be used to provide a computationally efficient overview of the most important features present in the nodes that make up a graph. In my framework the gradient/feature based explanation is used to identify the most important features in the previously identified most important nodes and quantify their importance for the predictions through attribution values.

Model-level explanations

Model-level explanations provide explanations for a GNN that are not dependent on any particular input sample. They aim to provide a more general understanding of the model and its ability to elucidate generic behaviors. GNNInterpreter is an example of a model-level explainer that was considered for use in the current project. It provides explanations by learning a probabilistic graph distribution that reveals a class's discriminative pattern rather than explaining single predictions [27]. Both self-interpretable and model-level explanations were considered for this work but were ultimately left out. Given that one of the goals of the thesis was to provide a general framework for applying explainability to PPM, post-hoc explanations can ensure the explainability block will work well with any given prediction model and doesn't need to rely on the explanation method being a part of the model itself. Model-level explanations were left out because there were no proper implementations found for working with heterogeneous graph neural networks using these techniques and an extension of these methods for this particular use was outside the scope of the work [28] [24].

Counterfactual Explanations

Counterfactual techniques aim to explain a model by identifying the minimal change that could be performed on the input graph to generate a change in the model's prediction. They offer a good complement to factual explanations by expanding a user's understanding of why a particular prediction is being made with an explanation of why a prediction was made instead of a different one [29]. Like factual explanations, counterfactual explanations can also be separated into instance and model-level explanations; my work focuses entirely on instance level. Counterfactual explanations can further be broken down into three main groups: perturbation-based, heuristic-based, and search-based. Perturbation-based techniques, much like their factual counterparts, consist of modifying the existing graph by adding or eliminating edges in the graph until a change in the prediction made by the model is

achieved. The resulting graph is considered a counterfactual of the original input. Heuristic-based methods rely on a rigorous policy of modifying the input graph until they reach a valid counterfactual. Search-based techniques rely on using search methods within the counterfactual space to identify relevant candidates. The main challenge of this technique is either properly limiting the search space or employing sufficiently efficient search algorithms that the exponential search space is not a problem. An example of this methodology called NICE offers a general nearest-unlike-neighbor counterfactual algorithm for heterogeneous tabular data [30]. Another example of this methodology used in PPM is the LORELEY method which extends LORE (a counterfactual method that generates explanations using genetic algorithms) with control-flow constraints for realistic counterfactuals in predictive business process monitoring [10].

Explainability metrics

Given the width of available XAI options that exist it was also necessary to provide a way to provide a comprehensive evaluation of their effectiveness. While ground-truth metrics can be used to calculate an explainer's accuracy on a synthetic dataset real-world applications will rarely provide such information. To this end, metrics that can be used to assess a model's technique in the absence of a ground-truth were developed [11]. The assessment on an explainer's technique aims to evaluate how good the explainer is at generating explanations that humans can comprehend. Metrics such as fidelity, a measure of the impact the important nodes the explainer identified have on the model's output, and sparsity, which quantifies the proportion of features that are deemed significant by the explanation, can provide a concrete idea of the effectiveness of the chosen explainer [28].

Explainability for heterogeneous graphs

While there are many XAI options for use with GNNs the vast majority of the existing methods have no direct implementation for heterogeneous graph neural networks, focusing only on homogeneous graph structures [28] [24]. This severely limits the available options for the current work and emphasizes the need for developing specialized explainability techniques that can handle the complexity of heterogeneous graph neural networks. Aside from the chosen implementation, two other different methodologies were considered. The first was an implementation of HGExplainer. This explanation method extends the GNNExplainer mask/mutual-information paradigm to heterogeneous graphs via meta-path sampling. However, its lack of open-source implementation and its proposed scope being specifically for link-prediction/recommender-systems tasks, not general regression made it infeasible to apply directly to the project [31]. A second option that was investigated was HTGExplainer. This method extends the same mask/mutual-information paradigm seen in previous explanation methods to heterogeneous and temporal GNNs making it an ideal fit for PPMs. The model has a publicly available implementation which was investigated and confirmed to work. However, the original work is based on a DGL implementation, a port was attempted to this project's chosen framework of PyTorch Geometric but it was unsuccessful [32].

As discussed further below, the final chosen methodology is based on the work of Zhai et al.[33]; however, certain aspects of their implementation were changed to better suit the data and analysis of this thesis. While their paper's own presentation conceptually motivates a heterogeneous graph with several station types (e.g., tube, bus and bike stations), their released implementation [33] represents every station as a single PyTorch Geometric node type, with the station type instead one-hot-encoded directly into each node's feature vector rather than modeled as

a distinct graph object; heterogeneity in their released graph is realized only through five edge types corresponding to road categories. This thesis's OCEL-derived graph, by contrast, models every event and object type as its own distinct node type, connected through three distinct categories of relation (object-to-event, event-to-event, and object-to-object) that reflect process semantics rather than physical geography. Given this difference, the ablation analysis, which in their work is a global, feature-level technique, was replaced by a perturbation-based explanation combining exhaustive LOO to identify the key elements in the structure and Shapley Values to quantify their impact at the instance level. Gradient/feature-based explanations are adopted as a second component to identify the important features within the most important nodes in the graph. The counterfactual implementation is adapted taking this structural difference in mind. Additionally a fourth component is added to the explainability layer in the form of metrics to assess the effectiveness of the explanations provided.

Table 2.1 summarizes the methods surveyed in this section across the dimensions most relevant to this thesis's setting. The table indicates whether a method natively supports heterogeneous node types, whether it can attribute importance to edges (not just nodes), whether it offers factual and/or counterfactual explanations, whether it has been applied to temporal/process data, whether an open-source implementation exists, and how each was ultimately treated in this project.

Methodological template

Of the eight methods surveyed, Zhai et al.'s [33] is the only one that natively supports heterogeneous graphs while also offering open-source availability and coverage of both factual and counterfactual explanations. Although their work was applied to an urban land-use inference domain rather than to predictive process monitoring, its heterogeneous graph transformer architecture and explanation coverage make it the closest existing template to my thesis's needs. The remaining methods, on the other hand, either target only homogeneous graph tasks, non-object-centric predictive process monitoring, or non-graph domains entirely. Because of this, the explainability structure set forth by Zhai et al. [33] in order to provide post-hoc explanations on a heterogeneous GNN framework was chosen as this thesis template. This methodology provides end users with explanations by focusing on three types of XAI methods: ablation analysis, feature attribution methods and counterfactual explanations. Their explanation methodology provides a holistic view of the information provided to the model by quantifying the importance the model is assigning to each feature and the changes that could be made to a given input in order to change the output.

Table 2.1: Comparison of the GNN-explainability methods surveyed in this section.

Method	Node types	Edge importance	Explanation type	Temporal	Open-source	Disposition in this thesis
GNNExplainer [25]	Heterogeneous via per-type mask workaround	× on heterogeneous graphs	Factual	×	✓	Tested; retained as comparison-only baseline
PGExplainer [26]	Homogeneous	Not evaluated	Factual	×	✓	Tested; rejected – heterogeneous integration too costly
GNNInterpreter [27]	No heterogeneous implementation found	— (model-level)	Factual, model-level	×	Not confirmed	Reviewed only; rejected – model-level scope, no heterogeneous support
NICE [30]	N/A – tabular, not graph	—	Counterfactual	—	Not stated	Reviewed only; background context
LORELEY [10]	N/A – sequence-based	—	Counterfactual (+ factual rules)	Implicit (sequential)	Not stated	Adapted as this thesis's counterfactual template, in a regression setting with graph dissimilarity instead of Levenshtein distance
HGExplainer [31]	Heterogeneous (meta-path)	Not confirmed	Factual	×	×	Reviewed only; rejected – no open-source implementation, link-prediction/recommender-systems scope only
HTGExplainer [32]	Heterogeneous	Not confirmed	Factual	✓ (only method explicitly temporal-capable)	✓, DGL only	Tested in DGL, confirmed working; rejected – PyTorch Geometric port unsuccessful
Zhai et al. [33]	Heterogeneous in concept (paper); released implementation uses a single PyTorch Geometric node type, station type one-hot-encoded into node features	Partial – factors into counterfactual dissimilarity, not per-instance masks	Both (factual + counterfactual + ablation)	×	✓, vendored	**Chosen as this thesis's template**, extended to true multi-node-type graphs

2.5 Summary and proposed framework

In summary, given the identified gaps in explainability research for predictive process monitoring I propose an explainability framework that adapts and extends the work set forth by Zhai et al. The framework is composed of four components: a perturbation-based explanation, a gradient/feature-based explanation, a counterfactual explanation and a set of metrics to assess the explanation provided to the end user. This framework is designed to be model-agnostic and expand on current state of the art predictive process monitoring methods that use heterogeneous graph neural networks to provide KPI estimations.

3 Preliminaries

In this chapter I introduce the reader to the preliminary concepts that will be used in the rest of the thesis as well as the datasets chosen for this research and their associated processing. The goal of this chapter is to provide readers with a complete view of the data processing pipeline that was performed so it may be replicated. In Section 3.1 I introduce concepts from the literature on traditional event logs, KPIs, and a formalization of the problem of predictive process analytics. In Section 3.2 I report on the characteristics of object-centric event logs while in Section 3.3 I expand on the use of predictive analytics on object-centric processes. Afterwards, in Section 3.4 I give a rundown of existing explainability methods and a formal definition of the ones chosen for this thesis' framework.

3.1 Single-id event logs and predictive process monitoring

I present the following definitions based on the formalizations provided by Galanti et al. [2].

Definition 1 (Single-id Event Log [2]) *A single-id event log is a tuple:* $L = (E, Tr, A, C, \pi_{time}, \pi_{act}, \pi_{att})$.
Let $\mathcal{E}$ *be the universe of events,* $\mathcal{A}$ *be the universe of activities,* $\mathcal{C}$ *be the universe of attributes,* T *be the universe of event timestamps, and* $\mathcal{AV}$ *be the universe of attribute values.*
Let $A \subseteq \mathcal{A}$ *be a set of activities and* $C \subseteq \mathcal{C}$ *be a set of attributes such that* L *is composed of:*

- *a set* $E \subseteq \mathcal{E}$ *of events,*
- *a set* $Tr \subset E^*$ *of traces, i.e. sequences of events.*
- *a time mapping* $\pi_{time} : E \to T$ *associating each event to that event's timestamp,*
- *an activity mapping* $\pi_{act} : E \to A$ *associating each event to that event's activity and,*

- *an attribute mapping* $\pi_{att} : E \times C \nrightarrow \mathcal{AV}$ *associating each event to that event's attribute assignment.*

The goal of predictive process monitoring methods is to predict the value of a key performance indicator. Therefore, it is necessary to provide a definition of the KPI function.

Definition 2 (KPI Function [2]) *Let* $L = (E, Tr, A, C, \pi_{time}, \pi_{act}, \pi_{att})$. *be a single-id event log.*
Let $\mathcal{W}_k$ *be a set of possible KPI values. A KPI is a function* $\mathcal{T}_L : Tr \times \mathbb{N} \nrightarrow \mathcal{W}_k$ *such that, given a trace* $\tau \in Tr$ *and an integer index* $i \leq |\tau|$, $\mathcal{T}_L(\tau, i)$ *returns the KPI value of* τ *after the occurrence of the first* i *events.*

My KPI definition assumes it to be computed a posteriori, once the entire process execution has been completed and a complete set of traces is available. KPI definitions aim to be generic to account for all possible domains. The KPI value may be updated after each execution or it may only be known after the completion. Here, I present an example of a KPI function:

Remaining Time The KPI function provides a measure of the amount of time left before the process execution is complete. It is defined as the difference between the timestamp of the final event in the process execution, e_n and the last occurred event e_i: $\mathcal{T}_{remaining}(\tau, i) = \pi_{time}(e_n) - \pi_{time}(e_i)$

I will now formalize the definition of the prediction problem on single-id event logs.

Definition 3 (Prediction Problem on Single-ID Event Logs [2]) *Let* $L = (E, Tr, A, C, \pi_{time}, \pi_{act}, \pi_{att})$. *be a single-id event log. Let* $\mathcal{T}_L$ *be a KPI function defined for the process execution.*
Let $\tau = \langle e_1, ..., e_k \rangle$ *be a trace of a running process execution which will eventually complete as* $\tau_T = \langle e_1, ..., e_k, e_{k+1}, ..., e_n \rangle$
The prediction problem can be formulated as forecasting the value of $\mathcal{T}_L(\tau_T, i)$ *for all* $k < i \leq n$.

3.2 Object-centric event logs

Object-centric event logs are extended versions of the single-id event logs defined previously meant to provide a representation that more closely resembles real-world applications. In a typical object-centric event log each event may refer to one or multiple objects, as opposed to single-id event logs which assume each object is of the same type and each event is associated with exactly one object.

Table 3.1: Object-Centric Event Log With Associated Objects

Id	Activity	Timestamp	Order	Items	Packages	Price	Weight
place_o-990001	Order Placement	2023-04-03 12:08	o-990001	i-880001, i-880002, i-880003		2029.00	
confirm_o-990001	Order Confirmation	2023-04-04 22:51	o-990001			2029.00	
pick_i-880001	Item Selection	2023-04-05 03:18		i-880001		2029.00	
pick_i-880002	Item Selection	2023-04-05 03:18		i-880002		2029.00	
pick_i-880003	Item Selection	2023-04-05 03:18		i-880003		2029.00	
reminder_o-990001_1	Payment Reminder	2023-04-05 15:31	o-990001			2029.00	
pay_o-990001	Order Payment	2023-04-06 15:08	o-990001			2029.00	
create_p-660003	Package Creation	2023-04-11 14:37		i-880001, i-880002, i-880003	p-660003	2029.00	1.20
send_p-660003	Sending of Package	2023-04-11 19:37		i-880001, i-880002, i-880003	p-660003	2029.00	1.20
deliver_p-660003	Package Delivery	2023-04-13 04:57		i-880001, i-880002, i-880003	p-660003	2029.00	1.20
place_o-990002	Order Placement	2023-04-03 16:36	o-990002	i-880004		94.99	
confirm_o-990002	Order Confirmation	2023-04-05 03:20	o-990002			94.99	
pick_i-880004	Item Selection	2023-04-05 03:18		i-880004		94.99	
pay_o-990002	Order Payment	2023-04-05 17:13	o-990002			94.99	
create_p-660005	Package Creation	2023-04-11 19:21		i-880004	p-660005	94.99	0.28
send_p-660005	Sending of Package	2023-04-12 00:21		i-880004	p-660005	94.99	0.28
deliver_p-660005	Package Delivery	2023-04-13 09:41		i-880004	p-660005	94.99	0.28

Table 3.1 shows an example of an object-centric event log for a synthetic order management database meant to emulate the placement of orders composed of multiple items, the creation of packages and the delivery of the products. The event log consists of 4 object types: Orders, Customers, Items, and Packages, each with its own identifier. These 4 objects are a representative subset of the full collection of object types present in the dataset. The first object type, Orders, identifies the order that is being placed. Customers refers to the person or entity that placed the order; in this dataset the customer identifier is the customer's full legal name (In order to save space Customers is omitted as a table column and noted here instead: order o-990001 was placed by *Balkan Minerals d.o.o.* and order o-990002 by *Baltic Wave Energies Ltd*). Items identifies the items, or products, that form the order that must be delivered. Finally, Packages refers to the physical package that is created to house the items as they're shipped to their respective customers. The entire process may be complicated by the appearance of events such as Item Out of Stock or a Failed Delivery indicating that the package could not be delivered to the customer. Some events are associated with a single object identifier, but most events are related to two or more objects. Objects can be related to each other to enable synchronization and data exchange. These relationships can be of many-to-many or many-to-one in nature. Each event is also associated with the attributes, or features, of each related object. Orders have an associated price which quantifies the cost of the order that must be paid by the customer while packages have an associated weight. This is only an illustrative subset of the full set of features associated with each event in an actual process execution.

The following formal definitions are adapted from the work of Adams et al. [3] as well as de Leoni & Volpato [5].

Definition 4 (Object-Centric Event Log [3] [5]) *Let T be the universe of event timestamps. An object-centric event log is a tuple $L = (E, O, OT, A, C, AV, CT, \pi_{atype}, \pi_{otype}, \pi_{act}, \pi_{time}, \pi_{eatt}, \pi_{oatt}, E2O, O2O, \pi_{trace})$ such that:*

- *Set $E \subseteq \mathcal{E}$ where $\mathcal{E}$ is the universe of events,*
- *set $O \subseteq \mathcal{O}$ where $\mathcal{O}$ is the universe of objects,*
- *set $OT \subseteq \mathcal{OT}$ where $\mathcal{OT}$ is the universe of object types,*
- *set $A \subseteq \mathcal{A}$ where $\mathcal{A}$ is the universe of activities,*
- *set $C \subseteq \mathcal{C}$ where $\mathcal{C}$ is the universe of attributes,*
- *set $AV \subseteq \mathcal{AV}$ where $\mathcal{AV}$ is the universe of attribute values,*
- *set $CT \subseteq \mathcal{CT}$ where $\mathcal{CT}$ is the universe of attribute types,*
- *$\pi_{atype} : C \cup AV \to CT$ is a function associating an attribute name or value to its corresponding type*
- *$\pi_{otype} : O \to OT$ is a function that assigns one object type to each object instance*
- *$\pi_{act} : E \to A$ is a function associating each event to its corresponding activity*
- *$\pi_{time} : E \to T$ is a function associating each event to its corresponding timestamp*
- *$\pi_{eatt} : E \times C \nrightarrow AV$ is a function that assigns values to event attributes*
- *$\pi_{oatt} : O \times C \nrightarrow AV$ is a function that assigns values to object attributes*
- *$E2O \subseteq E \times \mathcal{Q} \times O$ where $\mathcal{Q}$ represents the universe of qualifiers describing relationships among entities,*

- $O2O \subseteq O \times \mathcal{Q} \times O$ *where* $\mathcal{Q}$ *represents the universe of qualifiers describing relationships among entities,*
- $\pi_{trace} : O \to E^*$ *is a function that maps each object to a sequence of events such that for every object* $o \in O, \pi_{trace}(o) = \langle e_1, ..., e_n \rangle$ *where* $\forall_i \in \{1, ..., n-1\}\ \pi_{time}(e_i) \leq \pi_{time}(e_{i+1})$ *and* $\forall_i \in \{1, ..., n\}\ \exists_s \in \mathcal{Q} : (e_i, s, o) \in E2O$ *where* $\mathcal{Q}$ *represents the universe of qualifiers describing relationships among entities and* $\{e_1, ..., e_n\} = \{e \in E \mid \exists s \in \mathcal{Q} : (e, s, o) \in E2O\}$ *is the complete set of events related to o.*

The relationships between objects in an OCEL can be expressed in the form of a graph built around the relationship that exists between events, objects to the events and the objects amongst themselves.

Definition 5 (Event-Object Graph [3] [5]) *Let* $L = (E, O, OT, A, C, AV, CT, \pi_{atype}, \pi_{otype}, \pi_{act}, \pi_{time}, \pi_{eatt}, \pi_{oatt}, E2O, O2O, \pi_{trace})$ *be an object-centric event log. The event-object graph* $OG_L = (O \cup E, I)$ *is an undirected graph where the set of undirected edges is defined as* $I = \{\{x, s, y\} \mid (x, s, y) \in E2O \cup O2O \vee (y, s, x) \in E2O \cup O2O\}$

To represent the complexity of the recently defined event-object graph it is necessary to use a structure that can properly handle different node and event relationships. That is why heterogeneous graphs were chosen. In line with Adams et al.'s work I induce my process executions through leading type extraction, where a process execution is constructed for each object of a chosen leading object type, which I define as my viewpoint. I can define the distance between two nodes n_1 and n_2 within OG_L as $dist : O \cup E \times O \cup E \to \mathbb{N}_0$ where, for each pair, n_1, n_2 of nodes, $dist(n_1, n_2) = \infty$ if the nodes are not connected.

Definition 6 (Process Executions [3] [5]) *Let* $L = (E, O, OT, A, C, AV, CT, \pi_{atype}, \pi_{otype}, \pi_{act}, \pi_{time}, \pi_{eatt}, \pi_{oatt}, E2O, O2O, \pi_{trace})$ *be an event log, let* $OG_L = (O \cup E, I)$ *be the corresponding event-object graph, let* $\mathcal{OT}$ *be the universe of object types, and let* $\mathcal{V} \in \mathcal{OT}$ *be a viewpoint. For each object* $o \in O$ *such that* $\pi_{otype}(o) = \mathcal{V}$*, a process execution for o is an undirected graph* (N, A) *where* $N = O' \cup E'$ *where* O' *is the set of objects assigned to o and* E' *is the set of events related to objects in* O'*: -* $O' = \{o'' \in O \mid \pi_{otype}(o'') \neq \mathcal{V} \wedge \forall_{o' \in O}\ \pi_{otype}(o') = \mathcal{V} \implies dist(o', o'') > dist(o, o'')\}$ *-* $E' = \{e \in E \mid \forall_{o' \in O}\ \pi_{otype}(o') = \mathcal{V} \implies dist(e, o') > dist(e, o)\}$ *Arcs are then defined as:* $A = \{\{e, \top, e'\} \mid \exists_{o^*} \in O'\ \langle e, e' \rangle \subseteq \pi_{trace}(o^*)\} \cup \{\{a, s, b\} \in I \mid a \in N, b \in N\}$

Each process execution derived from this definition is a connected component of the full OCEL graph. The dataset used for predictive analysis ends up as a list of distinct components rather than one interconnected graph. The use of a heterogeneous graph structure to represent process executions eliminates the need for feature aggregation. In past homogeneous representations, it was necessary to condense the information provided by interacting objects into the event nodes, the only node type present in the graph. By allowing for multiple node types each object and event can preserve its individual characteristics as well as provide information that may have been difficult to properly aggregate before.

As I stated in 3 the goal of a predictive process is to infer the value of a KPI function when the process execution is at an intermediate state. Therefore, the definition of the object-centric process prediction problem requires the introduction of the concept of a prefix of a process execution.

Definition 7 (Prefix of a Process Execution [5]) *Let $L = (E, O, OT, A, C, AV, CT, \pi_{atype}, \pi_{otype}, \pi_{act}, \pi_{time}, \pi_{eatt}, \pi_{oatt}, E2O, O2O, \pi_{trace})$ be an object-centric event log and T the universe of timestamps for the events in L. Let P be the universe of potential process executions from L. The function $Prefix : P \times T \to P$ returns a snapshot of process executions. Given a timestamp t and a process execution $p \in P$, $Prefix(p, t)$ returns a snapshot of p at time t. Formally, given a process execution $p = (N, A)$, $Prefix(p, t) = (N', A')$ where $N' = E' \cup O'$ with: - $E' = \{e \in E \cap N \mid \pi_{time}(e) \leq t\}$ - $O' = \{o \in O \cap N \mid \exists e \in \pi_{trace}(o) : \pi_{time}(e) \leq t\}$ - $A' = \{(a, s, b) \in A \mid a \in N', b \in N'\}$*

Given this definition a prefix obtained from a timestamp t includes in it only objects and events that have already occurred. Each prefix $\bar{p}$ is a subgraph of p and is still considered a process execution. With prefixes properly defined I will now provide a formal definition of object-centric KPIs and the problem of object-centric process prediction.

Definition 8 (Object-Centric KPI Function [5]) *Let $L = (E, O, OT, A, C, AV, CT, \pi_{atype}, \pi_{otype}, \pi_{act}, \pi_{time}, \pi_{eatt}, \pi_{oatt}, E2O, O2O, \pi_{trace})$ be an object-centric event log, T the universe of timestamps for the events in L and let P be the universe of potential process executions from L. An Object-Centric KPI is a function $K : P \times T \nrightarrow \mathcal{W}_{ko}$ such that, given any process execution $p \in P$, it returns a vector $K(p, t)$ of the KPI values at time t.*

Definition 9 (Prediction Problem on Object-Centric Event Logs [3] [5]) *Let P be the universe of potential process executions and p be a complete process execution, and let p' be the prefix of p at time t such that $p' = Prefix(p, t)$. The prediction problem on Object-Centric Event Logs can be formulated as computing the value of $K(p', t)$ where the computation of $K(p', t)$ may depend on the full execution of p beyond t. Therefore, its computation requires the estimation of key elements within the structure of p.*

3.3 Predictive Analytics on Object-Centric Processes

In this section I will explain how, following the approach for graph encoding first proposed by Adams et al. and explained in Section 2, the native object-centric presentation of OCELs can be maintained and used to obtain predictions. This approach is favored over a flattened tabular representation since it allows me to avoid problems such as deficiency, convergence, and divergence. Predictions will be obtained using graph-based predictive models, such as graph neural networks (GNNs), which are designed to operate directly on graph structured data, learning not just from the features present in the data itself but also from the underlying structure connecting the components.

The proposed GNN model is trained on a multiset of labeled examples known as the training set which is defined as set of pairs $(x, y) \in \mathcal{X} \times \mathcal{Y}$ where $\mathcal{X}$ encodes the independent variables (also known as features) with their values, and $\mathcal{Y}$ is the dependent variable which is encoded with the value I wish to predict.

The prediction technique also requires the definition of a trace-to-instance encoding function $\rho : P \to \mathcal{X}$ which maps each trace (or process execution) p, or a prefix p' of it, to an element $\rho(p) \in \mathcal{X}$.

The prediction model is then trained on a data set $\mathcal{D}$ created from an object-centric event log L. $\mathcal{D}$ is composed of a series of distinct pairs $(x, y) \in (\mathcal{X} \times \mathcal{Y})$, where $x = \rho(p')$ and $y = K(p', t)$ computed for each prefix p' of each trace $p \in P$. The resulting prediction model can be abstracted as an oracle function $\Phi_{\mathcal{D}} : \mathcal{X} \to \mathcal{Y}$ [6]

Although the use of GNNs for predictive process monitoring tasks using OCELs has provided mixed results regarding their accuracy when compared to more traditional models, the trade-off for maintaining the underlying graph structure is worth it in terms of explainability [6] [4]. By maintaining the original OCEL structure intact I can provide the explainability layer with a more complete view of the information driving the predictions and offer the end user a more holistic understanding of how it's being processed.

Graph neural networks (GNNs) are neural models that capture the dependence of graphs via message passing between the nodes of graphs. As I explained in Section 2.3 GNNs learn via a message-passing mechanism, built from two functions: an aggregation function and an update function. At each iteration, each node collects information from the neighborhood around it in the form of messages coming from its neighbors. As the iteration progresses each node embedding is updated with information from more distant reaches of the graph. At the first iteration ($k = 1$), each node embedding contains information from its 1-hop neighborhood. A k-hop neighborhood is defined as the set of nodes with *dist* value equal to k, where *dist* is calculated within each process execution used for training the model. At the second iteration, each node embedding contains information from its 2-hop neighborhood, and so on. Each node's embedding stores information that is both feature-based as well as structural, providing information about the graph itself. In the message-passing mechanism, each node has its message stored in a feature vector which gets updated with information provided by its neighbors. This aggregation process connects nodes and forms new feature vectors by incorporating updated information.

Definition 10 (Message Passing Mechanism in Graph Neural Networks [14]) *Let n be the number of iterations, u a given node, g the node's message, AGGREGATE and UPDATE differentiable functions, and $m_{\mathcal{N}(u)}^{(n)}$ be the aggregated message from u's graph neighborhood $\mathcal{N}(u)$* $g_u^{(n+1)} = UPDATE^{(n)}(g_u^{(n)}, AGGREGATE^{(n)}(g_v^{(n)}, \forall_v \in \mathcal{N}(u))) = UPDATE^{(n)}(g_u^{(n)}, m_{\mathcal{N}(u)}^{(n)})$

3.4 Explanation on process predictions

In this section I explain the XAI methods proposed to enhance explainability in predictive process monitoring on object-centric event logs: perturbation-based analysis, gradient/feature-based analysis, counterfactual analysis, and a set of metric assessments. As I explained in Section 2.4.1 these methods are motivated by a need to instill trust in the end users and stakeholders who would rely on the provided models to generate predictions on a set of KPIs. Perturbation-based analysis provides a way to identify the key structural elements of the input graph and provide an estimate of its impact on the predicted value. Gradient-based analysis provides a deeper look at the importance of specific attributes affecting a model's output. Counterfactual analysis enables users to explore the minimal changes that could lead to a desired outcome, such as the reduction of time remaining until a process's completion. Finally, the metric assessment allows the user to validate how effective the given methods are at providing reliable explanations on the given model, ensuring the end user can trust the information provided.

The methods chosen reflect the current state of explainability approaches available for use on heterogeneous graph models. As I explained in Section 2.4.3 other methods were investigated but were not chosen because of either a lack of a publicly available code implementation or an inability to translate the explanation model into the currently used framework.

3.4.1 Perturbation-based analysis

The first methods used in my framework to provide an explanation on the model are perturbation-based. These methods quantify the impact of each element in the graph by measuring the difference between the model's original prediction and the model's prediction with an element, or a set of elements perturbed, or changed, in some measurable way. Through an iterative approach these methods are able to construct a subgraph that contains all the most important elements used by the model to provide a prediction. My proposed framework applies perturbation-based analysis on the model through a combination of two approaches: exhaustive Leave-One-Out analysis used to identify the most important nodes and edges in the input graph and then Shapley Values to provide an estimate of the impact the identified nodes have on the model's prediction in a physically meaningful magnitude. As I explained in Section 2.4.3 while the usage of Shapley Values is hindered by its computational expense this can be mitigated by reducing the explanation space (the set of features of the input) to which these method needs to be applied using LOO. I will now formalize the Leave-One-Out and Shapley Value implementations used for explaining my heterogeneous models.

Definition 11 (Leave-One-Out Perturbation) *Let $p = (N, A)$ be a process execution with viewpoint node v, let Φ be the trained GNN model, let $X = \{x_j \mid v_j \in p\}$ be the set of feature vectors associated with the nodes of p, and let $\hat{y} = \Phi(p, X)$ be the baseline prediction for v. For a node $v_j \in p$ with feature vector $x_j \in X$, let X^{-v_j} be the perturbed feature set obtained by replacing x_j with the zero vector 0, leaving every other node's features and the structure of p unchanged. The Leave-One-Out shift of v_j is defined as $\delta(v_j) = \hat{y} - \Phi(p, X^{-v_j})$*

LOO functions by masking a particular feature, this equates to setting its associated feature values to 0. This is fundamentally different from actually removing the feature from the graph and restructuring its form around that ablation. While I considered this method for implementation I decided against it because such a structural change would provide explanations on a graph structure that did not really exist within the universe of process executions and the associated prefixes, meaning the explanations would provide no real value to the stakeholder.

Through the application of LOO to each graph feature I can obtain a reasonable estimation of the most impactful elements in the graph. Then, the true impact of the selected elements is calculated through the use of Shapley Value analysis.

Definition 12 (Permutation-Sampling Shapley Value [8]) *Let $p = (N, A)$ be a process execution with viewpoint node v, let Φ be the trained GNN model, and let $X = \{x_j, v_j \in p\}$ be the set of feature vectors associated with the nodes of p. Let $M \subseteq N$ be a candidate set of elements whose importance is to be attributed. For a coalition $S \subseteq M$, let X^S denote the feature set obtained from X by replacing the feature vector of every element in M/S with the zero vector 0, leaving the features of elements outside M and the structure of p unchanged, then define the value function $val(S) = \Phi(p, X^S)$ Let $val(M) = \hat{y}$ be the real baseline prediction of the model and $val(\emptyset)$ be the value once every element of M has been zeroed. The exact Shapley Value of $v_j \in M$ is: $\varphi(v_j) = \frac{1}{|M|!} \sum_{\pi \in \Pi(M)} [val(Pre_\pi(v_j) \cup \{v_j\}) - val(Pre_\pi(v_j))]$ Where $\Pi(M)$ is the set of all permutations of M and $Pre_\pi(v_j)$ is the set of elements preceding v_j in permutation π.*

The same overall definition can be applied when M denotes a candidate set of arcs instead of nodes in order to quantify the importance of edges in the input graph.

Since evaluating all $|M|!$ permutations of the input graph is too computationally expensive, I approximate $\varphi(v_j)$ using Castro et al.'s ApproShapley algorithm, formalized as:

Definition 13 (ApproShapley Algorithm [8]) $\hat{\varphi} = \frac{1}{R}\sum_{r=1}^{R}[val(Pre_{\pi_r}(v_j)\cup\{v_j\}) - val(Pre_{\pi_r}(v_j))]$ *where* $\pi_1, ..., \pi_R$ *are R permutations of M sampled uniformly at random, requiring* $R \cdot |M|$ *evaluations of val.*

Through the application of LOO and Shapley Values my methodology can identify the key nodes and edges present in the graph and provide the end user with an explanation subgraph for the process execution, composed entirely of the elements that impact the model's prediction.

As I explained in Section 2.4.3 LOO and Shapley Values are the core explanation methods used for my framework, however, GNNExplainer is also available as a standalone comparison point to evaluate how good the chosen methods are against a different implementation.

GNNExplainer is an explainability method that computes an explanation by formulating a mean field variational approximation and learning a real-valued graph mask which selects the important subgraph of the GNN's computation graph. The given explanation is therefore a rich subgraph of the entire graph the GNN was trained on, which maximizes the mutual information with the model's prediction.

Definition 14 (GNNExplainer [25]) *Let v be a node,* $p_s \subseteq p$ *be the subgraph of process execution p that contains all the important information for the GNN's prediction,* $X_s = \{x_j \mid v_j \in p_s\}$ *be the associated features of* p_s *where* v_j *ranges over the nodes contained in* p_s*, Y is a random variable given by the conditional distribution* $P_\Phi(Y \mid p, X)$ *learned by GNN model* Φ *and* $H(Y)$ *is the entropy term of the random variable.* $\max_{p_s} MI(Y, (p_s, X_s)) = H(Y) - H(Y \mid p = p_s, X = X_s)$

MI thus quantifies the change in the probability of prediction $\hat{y} = \Phi(p, X)$ when a viewpoint node v's computational graph is limited to the explanation subgraph p_s and its node features are limited to X_s.

Although GNNExplainer is able to identify which nodes are structurally relevant to the model's prediction in a more efficient manner, the value of its learned mask is a $[0,1]$-bounded attention weight rather than a meaningful measure of the change in the prediction's outcome. Additionally, its current heterogeneous implementation cannot account for edge importance when applied on a natively heterogeneous model like the chosen HGT architecture.

In the first implementations of the explanation framework the perturbation-based explanation component was based around GNNExplainer identifying the key elements in the input graph and LOO being used to quantify the values. However, this approach was limited due to the problems outlined in this section. First, GNNExplainer's inability to account for edge importance in heterogeneous models meant that it could not be used to identify the most important edges in the graph, therefore the goal of reducing computing cost by limiting the explanation space with the more efficient model worked only on nodes. This meant that LOO had to be run exhaustively on the model anyway.

The second limitation was in the way LOO quantified the feature's impact on the prediction. As seen in this section, LOO is one of the permutations of the graph that are accounted for through the usage of Shapley Values. Since it evaluates the effect of a singular element having its feature values set to 0 through masking while keeping

everything else in the graph intact it can provide a reasonable understanding of a feature's overall impact on the prediction. However, its focus on a singular feature leads to it downplaying the impact of interconnected elements across the graph. The two methods agree only when features contribute independently. If features can substitute for each other or amplify each other they diverge.
For example, if event nodes and Order nodes both encode similar timing information, LOO will underestimate both because each masks the other whereas the Shapley Values method would be able to properly identify the magnitude of the impact each element has on the prediction. Because of this, the framework was updated to the current LOO as identifier and Shapley Values as quantifier method with GNNExplainer retained as a comparison point only.

3.4.2 Gradient/feature-based analysis

The second class of techniques used to explain the predictions of my models are gradient/features-based methods. These techniques quantify an individual feature's importance on the model's prediction through linear feature attribution scores.

Let process execution $p = (N, A)$ with $N = O' \cup E'$, $NT = OT \cup ev$ be the set of node types in process execution p, where ev denotes the event type. For each node type t let $N_t \subseteq N$ be the nodes of type t such that $N_t = \{o \in O' \mid \pi_{otype}(o) = t\}$ for $t \in OT$, and $N_t = E'$ for $t = ev$. Let $X = \{X_t\}_{t \in NT}$ where $X_t \in \mathbb{R}^{|N_t| \times |\mathcal{F}_t|}$ with $\mathcal{F}_t$ as the feature set associated with node type t. I can formalize the feature attribution scores as follows.

Definition 15 (Feature Attribution Scores [33]) *Given a target node $v_i \in N$ and a target output $\hat{y}_i = \Phi(p, X)[v_i]$, my goal is to compute the attribution scores S where $S = \{S_t\}_{t \in NT}$ and $S_t \in \mathbb{R}^{|N_t| \times |\mathcal{F}_t|}$ estimating the importance of every node and feature combination. Thus, the element $S_t[k, l]$ of the attribution score matrix provides an estimate of how much the $l-th$ feature of the $k-th$ node of type t, $X_t[k, l]$, affected the output $\hat{y}_i$.*

In my work I use two different gradient/feature-based methods for calculating the importance of each feature: Gradient · Input and Integrated Gradients.

Definition 16 (Gradient · Input Explanation [33]) *Given a trained model Φ, a process execution $p = (N, A)$, a target node $v_i \in N$ and a target output $\hat{y}_i = \Phi(p, X)[v_i]$ the Gradient · Input feature attribution scores are defined as $S_t^{G \cdot I}[k, l] = X_t[k, l] \frac{\partial \hat{y}_i}{\partial X_t[k,l]} = X_t[k, l] \frac{\partial \Phi(p,X)[v_i]}{\partial X_t[k,l]}$.*

The main advantages of the Gradient · Input method are its conceptual simplicity, its computational efficiency and its ability to provide an accurate capture of the behavior of the model around an input. However, its attributions may be unreliable if the function of the considered model saturates for a given input or if it is highly irregular and results in a problem of shattered gradients. Because of these problems I also chose to use the second method, Integrated Gradients, which is defined as follows.

Definition 17 (Integrated Gradients Explanation [33]) *Given a trained model Φ, a process execution $p = (N, A)$, a target node $v_i \in N$, a target output $\hat{y}_i = \Phi(p, X)[v_i]$ and a reference feature set $\overline{X} = \{\overline{X_t}\}_{t \in NT}$ the Integrated Gradients feature attribution scores are defined as $S_t^{IG}[k, l] = (X_t[k, l] - \overline{X_t[k, l]}) \cdot \int_{\alpha=0}^{1} \frac{\partial \Phi(p, \overline{X} + \alpha \cdot (X - \overline{X}))[v_i]}{\partial X_t[k,l]} d\alpha$.*

The Integrated Gradients method solves the problems of the Gradient · Input method by not only considering the gradient at the explained input instance, but rather, by calculating an aggregate gradient on the path from some suitable reference to the current input. In my experiments, I use the baseline feature set $\overline{X}$ obtained by zeroing every feature of the same graph p, based on recommendations shown in the work of Sundararajan et al. [9].

3.4.3 Counterfactual explanations

The previous two methods shown are factual explanations which seek to provide explanations to the end user by identifying the input parts that most impact the model's predictions. Counterfactual explanations, on the other hand, seek to identify the minimal changes that would be required on an input graph for the predicted output value to be a different, more desirable one. For graph data, counterfactual explanations usually take the form of subgraphs or modified graph inputs due to the connectivity in the nodes.

Zhai et al. [33] define a 1-hop complete subgraph as a shape for the counterfactual explanation. This is due to their dataset consisting of one large, connected graph. I do not need to provide such an explanation because my dataset is composed of a list of subgraphs (prefixes) of each process execution.

Since the goal of a counterfactual explanation is to identify the minimum change that an input graph would require to change its prediction a proper counterfactual subgraph would be the one that is least dissimilar to the input graph since it would require the least amount of effort to change. Dissimilarity can be calculated in a variety of ways, here I adapt the dissimilarity function provided by Zhai et al. for use with my dataset.

Definition 18 (Subgraph Dissimilarity [33]) *Let $p = (N, A)$ and $p' = (N', A')$ be two process executions with node-type sets $N_t \subseteq N$, $N'_t \subseteq N'$ and feature matrices X_t, X'_t where $NT = OT \cup ev$. Let $\pi_{ntype} : N \to NT$ extend π_{otype} to event nodes, such that $\pi_{ntype}(n) = \pi_{otype}(n)$ for $n \in O'$ and $\pi_{ntype}(n) = ev$ for $n \in E'$.*
And let $ET_p = \{(\pi_{ntype}(a), s, \pi_{ntype}(b)) \mid \{a, s, b\} \in A\}$ be the set of edge types occurring in p, with $ET_{p'}$ as its analogous definition for p'. The dissimilarity between p and p' is $D(p, p') = d_{feat}(p, p') + d_{type}(p, p') + d_{edge}(p, p') + d_{struct}(p, p')$

Since each element of the original definition needed to be adapted to better account for the fact that my process executions contain multiple node types, with multiple of them possibly repeating in the same trace I decided to give each element its own proper definition rather than overload definition 18.

A key difference between Zhai et al.'s work and mine is in their definition of nodes. While their paper conceptually motivates a heterogeneous graph with several station types, their released implementation [33] represents every node as a single PyTorch Geometric node type, with the station type one-hot-encoded directly into its features rather than modeled as a distinct graph object. This means that, at the implementation level, their graphs retain the same overall node structure regardless of station type. In order to account for this difference I treat every node of each shared type as its own seed, allowing me to also take into account multiple instances of the same node type in a process execution.

Definition 19 (Feature Dissimilarity Between Graphs [33]) *For each type t with $N_t, N'_t \neq \emptyset$ and each node $k \in N_t$, let*

- $\ell_t(k) = \frac{1}{|N'_t|} \sum_{j \in N'_t} \frac{||X_t[k,:] - X'_t[j,:]||_2}{\overline{||X'_t||} + ||X_t[k,:]||_2 + \varepsilon}$

- $c_t(k) = 1 - \frac{1}{2}(1 + \frac{1}{|N'_t|}\sum_{j \in N'_t} \cos(X_t[k,:], X'_t[j,:]))$ *where* $\overline{||X'_t||}$ *is the mean row-norm of* X'_t *and* ε *is a small constant preventing division by 0.*

$$d_{feat}(p,p') = \frac{1}{|NT_{shared}|}\sum_{t \in NT_{shared}} \frac{1}{|N_t|}\sum_{k \in N_t} \frac{\ell_t(k) + c_t(k)}{2} \text{ where } NT_{shared} = \{t \in NT \mid N_t \neq \emptyset \wedge N'_t \neq \emptyset\}$$

Definition 20 (Node Type Dissimilarity Between Graphs [33]) *A multiset-Jaccard distance over node-type counts.* $d_{type}(p,p') = 1 - \frac{\sum_{t \in NT} \min(|N_t|, |N'_t|)}{\sum_{t \in NT} \max(|N_t|, |N'_t|)}$

Another difference between my work and Zhai et al.'s is the edge types present in their data. While their work presumed their edge types form an ordinal scale my edge types are all categorical, and thus have no natural ordering. Therefore, I defined the distance in the same way as the node type dissimilarity, using a multiset-jaccard distance.

Definition 21 (Edge Type Dissimilarity Between Graphs [33]) $d_{edge}(p,p') = 1 - \frac{\sum_{r \in ET_p \cup ET_{p'}} \min(m_r, m'_r)}{\sum_{r \in ET_p \cup ET_{p'}} \max(m_r, m'_r)}$ *where* m_r, m'_r *are the number of edges of type r in p and* p' *respectively.*

A final difference is that Zhai et al. define the structure dissimilarity metric as the unnormalized absolute difference in node count between the two graphs. Since this is already given by the node type dissimilarity I instead define this dissimilarity over total edge count, normalized by the larger graph's edge count for scale invariance.

Definition 22 (Structure Dissimilarity Between Graphs [33]) $d_{struct}(p,p') = \frac{||A| - |A'||}{\max(|A|, |A'|, 1)}$

Each component is bounded in $[0,1]$, so $D(p,p') \in [0,4]$. This dissimilarity is not a true distance metric as it is not symmetric: $D(p,p') \neq D(p',p)$, because the feature component treats the nodes of the first argument as seeds compared to the second.

3.4.4 Explanation-quality metrics

As I explained in Section 2.4.3 an explanation's effectiveness can be evaluated through the use of metrics. These metrics are designed to not rely on the existence of a ground-truth explanation against which they can be compared, since real-world applications will typically not have them. I will now provide a formalization of the metrics used to evaluate the explanations provided by my methodology.

Fidelity is a metric used to assess whether an explanation's detected input node features are capable of describing the information. In theory, if a model is provided with a version of the input graph with the features identified as important removed it should show a considerable drop in accuracy. This is represented by the $Fidelity^+$ value. On the other hand, if the identified features are the only elements of the graph that are provided then the model's prediction should remain fairly consistent. This measure is referred to as $Fidelity^-$. Characterization is a value bounded in $[0,1]$ which provides an overview of the relationship between the two fidelity values. A characterization value closer to 1 indicates an overall more reliable explanation for the model. The following formalization adapts the fidelity metric surveyed by Nandan et al. [24] to this thesis's regression setting.

Definition 23 (Fidelity Metric of an Explanation [24]) *Let m denote the explanation for a model Φ, let p^{-m}, X^{-m} denote a process execution p with m's features zeroed out, and p^m, X^m, denote p with everything except m zeroed out.* $Fidelity^+(m) = |\hat{y}_i - \Phi(p^{-m}, X^{-m})[v_i]|$ $Fidelity^-(m) = |\hat{y}_i - \Phi(p^m, X^m)[v_i]|$ $Characterization(m) = \frac{Fidelity^+(m)}{Fidelity^+(m)+Fidelity^-(m)}$

Sparsity is used to quantify the proportion of features that are deemed significant by the explanation. Sparsity theory proposes that efficient explanations ought to be succinct, emphasizing only the key input elements and overlooking the remainder. Ideally, the sparsity value of an explanation would be high, indicating that the explanation is more effective in identifying the most significant features of the input while discarding those that are unimportant. This measure provides a way to quantify the overall conciseness and interpretability of the explanations provided. The following formalization adapts the sparsity metric proposed by Stevens et al. [11] to this thesis's node/edge-level explanation setting.

Definition 24 (Sparsity Metric of an Explanation [11]) *Let $p = (N, A)$ be a process execution and m denote the explanation with $m_n \subseteq N$ and $m_A \subseteq A$ denoting the nodes and edges respectively that m identifies as important. The node sparsity of m is* $Sparsity_N(m) = 1 - \frac{|m_N|}{|N|}$
and the edge sparsity of m is $Sparsity_A(m) = 1 - \frac{|m_A|}{|A|}$ *if $|A| > 0$, undefined otherwise.*

4 Models

In this chapter I discuss the experimental setup for the predictive models used for testing the explainability framework. I begin by outlining the datasets used as well as the processing that was applied to them. Afterwards, I provide an overview of the pre-processing done to ensure the chosen predictive model was working with the best possible hyperparameters and the comparison made against other baseline models to validate the chosen architecture. Finally, I explain the explainability framework that was applied to the predictive model in order to provide explanations to a process stakeholder.

4.1 Datasets

4.1.1 Order Management Dataset

The first of the two datasets used for validating my proposed explainability framework was a synthetic OCEL 2.0 benchmark dataset named Order Management [34]. The general process pipeline in the dataset is as follows; activities are placed in parentheses at relevant points of the explanation.

The process begins when a customer places an order for different products (Place Order). Each product has a price (which can vary on different dates) and a weight. When a customer places an order it is assigned to an employee to act as a sales-representative. The employees are tasked with confirming the customer's order (Confirm Order) as well as processing the payment (Pay Order) and reminding customers of payments that have not been received yet (Payment Reminder). Running parallel to this is the company's shipping process. After an order is received the order's items (unique identifiers for the products in each order) are checked against the company's stock. If an item is unavailable it is marked as out of stock (Item Out of Stock) and reordered by the warehouse employee (Reorder Item). Items that are ready for delivery are collected (Pick Item) for placement into packages that are addressed to a single customer. A package may contain items related to multiple orders, and an order's

Table 4.1: Summary of the datasets used in this thesis.

OCEL	Events		Objects			Traces
	Instances	Types	Instances	Types	Attributes	
Order Management	20801	11	10815	6	7	2000
Logistics	121913	14	47591	7	7	2000

volume may be distributed over multiple packages. After the readied items are collected into a package (Create Package) the package is picked up by a shipment employee for transport (Send Package). Finally, the package is shipped. Deliveries may fail repeatedly (Failed Delivery) until a successful delivery is achieved (Package Delivered).

The KPIs chosen to analyze in this dataset were Elapsed time from Orders to PackageDelivered, which identifies the time from the creation of the order until the delivery of the final package related to that order, and Elapsed time from Orders to PayOrder, which identifies the time from the creation of the order until the order is paid by the customer.

This dataset is composed of few, well-connected items so a model using only 2 layers was sufficient to create a path from the viewpoint to any other node type. This echoes the design decision of Smit et al., who used a similar model design, although their work was based around OCEL version 1.0 of this dataset [4].

4.1.2 Logistics Dataset

The second dataset I used to validate my proposed explainability framework was a different synthetic OCEL 2.0 benchmark dataset named Logistics [35]. The general process pipeline in the dataset is as follows; activities are placed in parentheses at relevant points of the explanation.

The process begins when a customer's order is registered at the company (Register Customer Order). After registration, a transport document in which the details of the rest of the process will be recorded is created (Create Transport Document). A logistics service provider is contacted to coordinate the transport of the ordered goods. Twice a week, the service provider sends a vehicle to a terminal to transport containers of ordered goods from the terminal to the seaport. Since the company depicted is not the only company that uses the provider's services available capacities may vary from vehicle to vehicle. Once the logistics provider receives the transport document, it books vehicles according to availability and priority (Book Vehicles). Once the dates for the transport of the goods are set, a container depot is contacted to reserve the required containers (Order Empty Containers). When a container vehicle's departure approaches the required goods are collected (Collect Goods). A truck is sent to the container depot to pick up the empty containers (Pick Up Empty Container). The truck is loaded with the collected goods (Load Truck), and it drives the full container to the terminal (Drive to Terminal). At the terminal, the container is picked up by a forklift and weighed (Weigh). The container may then be placed in the storage location at the terminal to await departure (Place in Stock). Finally, the container is taken to the loading bay and then moved onto the vehicle (Bring to Loading Bay, Load to Vehicle) where it departs at a set time (Depart). A container may miss a vehicle's departure and be rescheduled for the next possible vehicle (Reschedule Container).

The KPIs chosen to analyze in this dataset were Elapsed time from CustomerOrder to Depart, which identifies the time from the creation of the customer's order until the departure of the final container associated with it,

and Elapsed time from CustomerOrder to LoadToVehicle, which identifies the time from the creation of the customer's order until the container is loaded onto the vehicle.

This dataset differs from Order Management in its structure, with a higher quantity of object types and connections represented. Given this added complexity it was necessary to increase the chosen model's depth beyond 2 layers. This is due to the fact that a model at only 2 layers was incapable of creating a connection between the viewpoint node chosen and all the other node types present, causing the model to collapse to a near-constant predictor regardless of training budget. In order to find the proper layer depth the model was evaluated at 3, 4, and 5 layers for each KPI; layer depth 3 showed the best performance for Elapsed time from CustomerOrder to Depart, while Elapsed time from CustomerOrder to LoadToVehicle instead settled on layer depth 4.

4.1.3 Dataset Processing

In this section I expand upon the OCEL concepts introduced in Section 2.1.1 and formalized in Section 3.2. As stated there, the OCEL is generated by identifying the elements present in a process and the relationships that appear between them as events in the process are executed. In the case of my datasets the information related to objects, events and their relationships is found in an SQLite file containing a relational database of tables. These tables can be split into three main groups: tables relating to events, tables relating to objects and tables detailing the relationship between these elements. Table 4.1 provides a summary of the number of each instance present in each dataset. Tables relating to events provide a row by row accounting of all events of a certain type that occurred and the timestamp of the event's occurrence. Tables relating to objects provide similar information. For each object type they provide an OCEL identifier, a timestamp representing the time the object was created, a column indicating a change in one of the object table's columns, as well as columns for each attribute associated with the given object. For example, table object_packages contains information on each package type object in the database. This table consists of an identification column, an ocel_time column, an ocel_changed_field column, and a weight column, that provides the weight of the package.

Following traditional relational database conventions, the relationships between the elements in these tables are kept in separate tables labeled event_object and object_object (for event-to-object and object-to-object relationships respectively). All these elements can be extracted from the original database through SQL queries to generate the OCEL. Following the formalizations provided in Section 3.2 the OCEL creation process is as follows. A viewpoint object is selected from among the object types present in the database (for example, Order). For each object of the chosen type a sequence of related objects are identified through the use of the object_object table. Once all related objects are identified each event related to the objects is found by querying the event_object table. The resulting collection of events is then ordered according to their associated timestamp to create a basic event log. The log is then expanded upon by associating each related object of each event with its attributes, thus creating the OCEL.

It is important to note that while event-to-object and object-to-object relationships are directly provided by the database event-to-event relationships need to be inferred from the data. This process consists of identifying events with related objects that have a 1-hop distance between each other.

While the database provides a set of attributes for each object node, event nodes tend to have no related attributes. In order to enhance the information provided to the predictive model each node is feature-enhanced following the work of Adams et al. [3] by calculating object-centric features using information that is present in

the data but not explicitly stated. Such features may include values like waiting time which measures how long an event node had to wait before appearing in the process execution's graph, or an aggregate count of object types that have appeared so far in the process execution.

Given the wide array of values that can appear in a given node's features it is important to perform a process of standardization to ensure that large scale values, such as price, do not get assigned greater importance by the model when compared to smaller scale values such as weight. This standardization is also performed on the target value to facilitate faster convergence on the GNN's gradient descent-based algorithm. Standardization is performed using Z-score normalization. This process consists of calculating the mean and standard deviation of each feature value in the training set, then subtracting the mean from the value of each feature and then dividing by the standard deviation. This process ensures that each value is transformed to have a mean of 0 and a standard deviation of 1.

After these processes have been completed, the graph representation of each process execution and its associated prefixes is created from the OCEL's data. This results in the dataset being a series of subgraphs, as opposed to a single global schema of interconnected elements. Future work may follow a global approach, similar to the one proposed by de Leoni & Volpato [5], to see if it provides better results, but that implementation is outside the scope of this thesis.

4.2 Heterogeneous Graph Neural Network for Predictive Process Monitoring

In order to apply my proposed explainability layer I first needed to train a predictive model that could accurately predict values using the heterogeneous dataset that is created following the procedures outlined in Section 3.2. I began this process by defining a procedure for performing a temporal train/test split of the data. Afterwards, I decided on a model architecture that is well suited for the heterogeneous graph structure that the dataset is encoded in. By using a hyperparameter sweep I am able to identify the best hyperparameters to use in the chosen model architecture and could then perform a training and validation loop. Finally, I checked my results against baseline models to verify whether my chosen implementation is comparably effective against versions using other encodings, such as a homogeneous GNN or a traditional gradient boosted model on tabular data.

4.2.1 Train/test split of the datasets

The set of process executions and associated prefixes of the datasets all have an important temporal aspect related to them. The events in the processes follow a strict chronological order, indicating that some processes are completed long before other processes even begin. Therefore, it is important to ensure that there is no information leakage from future events into past ones when training the model. To ensure this, the train/test split of the data is performed ensuring the temporal separation between training, validation and test sets. In order to achieve this the universe of process executions was ordered chronologically according to the timestamp of their final associated event. The first 70% of the total process executions were selected for use in the training set. Any process executions outside this 70% that were considered active at the time of the completion of the final training set event (i.e. the process had begun before the completion of the final training set event and will finalize after the completion of the

final set event) were eliminated from the dataset to ensure that all the process executions present in the other sets were temporally separated from the training data. The process is repeated on the remaining process executions: the first 50% of the remaining ordered process executions are selected for the validation set, and all the process executions that remain after ensuring temporal separation again are assigned to the test set. This method inevitably results in process executions that are not taken into account, with the real train/test split percentages not equaling 100%. However, this is offset by the assurance of no information leakage between sets.

4.2.2 Model architecture choice and evaluation metrics

I considered two possible heterogeneous model architectures for this work. The first is Smit et al.'s [4] k-dimensional GNN. The other is Zhai et al.'s [33] HGT implementation. Since I am adapting Zhai et al.'s explainability layer, I decided to also work with their proposed heterogeneous model architecture; this way I could also offer a point of comparison against Smit et al.'s implementation. A major difference in both architectures is the way they handle the heterogeneous aspect of the input data. The k-dimensional GNN model transforms a homogeneous model design into a heterogeneous version through the use of PyTorch Geometric's to_hetero transformer. This transformation is achieved by duplicating the homogeneous model's message passing instance for each node type that exists in the heterogeneous input provided to the model. My chosen architecture, on the other hand, follows a strictly heterogeneous architecture built around heterogeneous graph transformers which are designed with per-type projection matrices and cross-type attention from the ground up.

To evaluate the performance of my chosen regression model I employed two metrics: Mean Absolute Error (MAE) and Root Mean Square Error (RMSE). These are widely recognized metrics for assessing the accuracy and predictive quality of regression models. Here I provide a definition of each metric, where n is the number of observations in the test set, y_i represents actual values and $\hat{y}_i$ represents predicted values.

- Mean Absolute Error (MAE): MAE quantifies the average absolute difference between the predicted values and the actual values. This measure provides insight into the overall error magnitude such that a lower MAE indicates smaller errors, and therefore better model performance.

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

- Root Mean Squared Error (RMSE): RMSE is a quadratic scoring rule that calculates the square root of the average of squared differences between the predicted and actual values. Same as MAE, lower RMSE values indicate better model performance and provide complementary insight by highlighting larger errors.

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

- Coefficient of Determination (R-Squared): R^2 is a statistical measure that indicates the proportion of the variance in the dependent variable that is predictable from the independent variables. It complements the other metrics by also providing a measure of how well the model handles the variability in the data. A higher value of R^2 indicates better model performance.

Table 4.2: Final chosen hyperparameters for this thesis's two datasets. Logistics is split into its two KPIs since, unlike Order Management, they do not share the same final configuration.

Hyperparameter	Order Management	Logistics (Depart)	Logistics (LoadToVehicle)
Number of epochs	30	100	100
Patience	4	10	10
Number of message-passing layers	2	3	4
Activation function	PReLU	PReLU	PReLU
Number of heads	2	2	2
Optimizer	Adam	AdamW	AdamW
Number of hidden dimensions	64	16	48
Learning rate	0.001	0.005	0.003
Weight decay	0.0	1e-05	1e-05

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2} \text{ where } \bar{y} \text{ is the mean of the actual values } y_i$$

4.2.3 Hyperparameter sweep

Having chosen the model architecture and the proper metrics for assessing my model's performance I needed to identify the most effective hyperparameter configuration per dataset and chosen KPI. In order to do this I performed a grid search over the model's number of hidden channels, and its learning rate. During this search I found that a mismatch between the weight decay used during the sweep (0.001) and the weight decay used in the actual regression testing (0.00001) led to the zeroing out of several node types' input projections entirely. The bug was identified and fixed during testing.

Aside from the values already mentioned the logistics dataset, given the problem discussed in Section 4.1.2, also searched for an optimal number of layers greater than 2, specifically among the values 3, 4, and 5. It's worth mentioning that any model that used only 1 layer for its architecture made any node type without a direct edge to the viewpoint unreachable for the model. The values used as possible choices for the search are informed by priors taken from Smit et al.'s [4] and de Leoni & Volpato's [5] reported findings on which values worked best for each of my chosen datasets.

4.2.4 Training and validation

With the model's hyperparameters set I was able to begin training and testing my model's predictions. I use an ADAM optimizer (AdamW for the two logistics KPIs, as detailed in Table 4.2), and an L1Loss function. In order to provide an accurate point of comparison to Smit et al.'s [4] work I aligned my testing parameters to the ones they reported on by setting up a testing environment with 30 epochs and a patience value of 4 epochs (i.e. if the model has not improved its prediction in 4 epochs the training loop ends). On the other hand, the logistics dataset

is not considered in Smit et al.'s work so the epoch and patience values did not need to align with any past results. Additionally, I found that at the selected patience level of 4 the predictive model undertrains. To combat this I increased the training epochs from 30 to 100 and its patience value from 4 to 10.

4.2.5 Baselines for comparison

In order to verify if the chosen heterogeneous architecture is truly more effective at performing predictions than past encodings I set up a comparison between the model and three baselines: a mean predictor used as a lower bound, Gradient Boosted Trees on a tabular encoding and a homogeneous GNN trained on a homogeneous representation of the dataset. These proposed baselines are adapted from Smit et al.'s work [4].

The homogeneous GNN model uses a GCN architecture with ReLU activation. The homogeneous representation of the input data is constructed as a graph with only the event nodes of the original heterogeneous graph and their relationships present. Each event node has all the features present in its heterogeneous representation as well as a target value taken from the viewpoint object node, thus allowing for a direct comparison between a feature-enhanced heterogeneous encoding versus a feature-enhanced homogeneous encoding.

Additionally, I wished to check whether the heterogeneous encoding provides a benefit for predictions over longer prefixes. My hypothesis was that the heterogeneous models would outperform the baselines significantly at greater prefix depths, becoming more precise the further along the process is. This is especially important for real world applications, as the predictions given by a model on a recently started process should be less accurate than the prediction it can provide on a much more mature one with more information to draw from.

4.3 Explainability Framework Design

As outlined in Section 2.4 my proposed explainability framework is composed of four distinct components meant to enhance an end user's understanding of the predictive model, the reasons behind its predictions, and the ways in which those predictions can be changed through changes in the input data.

4.3.1 Perturbation-based explanation (LOO + Shapley Values)

The first component of the explainability framework is a perturbation-based analysis performed using a combination of Leave-One-Out (LOO) methodology and Shapley Values. As explained in Section 3.4.1 LOO is used to identify the most impactful features in the input graph in order to reduce the explanation space where the full Shapley Values method needs to be applied. The stakeholder is given the ability to decide on the K number of most impactful elements they wish to view from the graph. Those elements are identified and then their impact is fully quantified with the Shapley Values method. The stakeholder is then presented with a series of bar graphs identifying the most important elements in a given process execution and their calculated impact on the prediction should they be removed as well as a table summary of the same information to ensure its proper understanding. Additionally, an explanation subgraph is constructed using the identified elements and also presented to the stakeholder. The ranking of node-importance obtained through this explanation method is also shown in comparison to GNNExplainer's ranking as a validation check. As outlined in Section 3.4.1 GNNExplainer's

learned mask is a [0,1]-bounded attention weight rather than a physically meaningful prediction-magnitude shift, and its heterogeneous implementation can't provide edge importance on a natively heterogeneous model like the chosen HGT architecture. Therefore, its role is limited to serving as only a comparison point.

This analysis can also be performed at an aggregate level, quantifying the average impact of each feature found in a collection of process executions as opposed to a single trace. The aggregate values are obtained by calculating the feature's impact on its respective trace then averaging it over the number of appearances of that particular feature. The stakeholder is shown the resulting analysis in the form of a bar graph and a table similar to the one used to present the single trace results in order to maintain consistency in the presentation.

It is important to note that the zero-vector substitution the LOO method uses to mask an element was itself validated against two other options: substituting the removed value with NaN or using a continuous soft mask like GNNExplainer. The first option was found to be untenable since using NaN led to a break in HGTConv's attention-weighted propagation. On the other hand, the second approach confirmed on a representative case that the current hard-zero design already is the fully-suppressed limit of a GNNExplainer-style soft mask.

4.3.2 Gradient/feature-based explanation (Gradient · Input and Integrated Gradients)

The second component of the explainability framework is used to complement the perturbation-based analysis of the first component by providing a finer-grained view of which features drive the important elements that component highlights. As explained in Section 3.4.2 this is done through the use of gradient-based methods applied across the model. At trace-level this analysis provides the stakeholder with a bar graph showing the attribution values for the top K attributes in the current trace that are impacting the prediction. While the attribution value gives a general understanding of the importance of each feature it carries no dimensional value so it may be difficult to interpret. In order to improve the stakeholder's understanding each feature selected through a gradient method is then used to perform a Shapley Value analysis in order to provide the stakeholder with an interpretable magnitude of the shift the feature can have on the prediction. This attribution can be performed with either Gradient · Input or Integrated Gradients to allow for validation of each method.

Similar to how the perturbation-based explanations are shown, at an aggregate level the component analyzes the average impact of each feature of every node of a given type in the test set over their respective model predictions. The identified elements are shown to the user, ranked by importance and side by side with a bar graph showing the quantified impact on the prediction obtained through aggregate Shapley Value analysis on those features.

4.3.3 Counterfactual explanation

The third component of the framework is the counterfactual explanation which is used to provide the user with a direct comparison between existing subgraphs in the dataset that are similar but have different predicted values. As formalized in Section 3.4.3 the analysis is broken into two parts: first, selecting the set of counterfactual candidates to compare against the factual graph and second, calculating the dissimilarity score between the factual graph and each possible candidate to identify the most similar graph with the desired prediction value.

The first part of the process adapts the work of Huang et al. [10] and their LORELEY model for predictive processes. Their work outlines a way to identify possible candidate subgraphs within the universe of process executions, however, their work is designed for a classification problem. Three steps were taken to adapt their work for my thesis's regression setting. First, a minimum-meaningful-difference threshold is set instead of a simple class filter. This way the end user can select the minimum value difference they would like to see between the subgraphs when comparing them. Second, a set of explicit prefix-depth-matching rules were applied so candidates are compared at similar points in their process lifetime rather than across arbitrary lengths. Finally, the end user is allowed to decide whether the candidate values should be lower or greater than the current graph's prediction, this way the process can be extended for use cases such as revenue analysis where a higher predicted value is the goal.

The second part adapts the dissimilarity metric used by Zhai et al [33] to calculate how similar each of the chosen candidate graphs are to the factual graph. After calculating the dissimilarity metric for each pair the candidate with the lowest dissimilarity metric (i.e. the most similar graph) is chosen as the counterfactual example to show to the stakeholder.

The stakeholder is then shown a graph representation of the factual graph as well as the chosen counterfactual. They're also shown a bar chart with the values of the four dissimilarity components that make up the final dissimilarity metric so they're able to identify which elements are driving the difference. Finally, the end user is provided with the specific elements that differ between each graph. With all these elements the stakeholder is able to identify the element driving the difference.

4.3.4 Fidelity metrics

The stakeholder is also presented with the fidelity metrics defined in Section 3.4.4, calculated for the perturbation-based explanations and the resulting subgraph. By reviewing the fidelity values, the resulting characterization value as well as the sparsity of the explanation the stakeholder is able to judge how effective the proposed explanation is at assessing the model's functionality [11].

5 Results

In this chapter I show the results of my thesis work, going from the initial dataset processing, through the model tuning, training and validation and finishing with the results of my proposed explainability framework applied to the given datasets.

5.1 GNN experiments

In the first part of this section I present the results related to the heterogeneous predictive model implemented. I begin by showing the results of the model's hyperparameter sweep. Then, I provide a comparison of its learning curve against the homogeneous implementation to better understand the effects of using a heterogeneous model instead of a homogeneous one. Finally, I provide a summary of the results of the model against all other baselines for a full comparison.

5.1.1 Hyperparameter sweep

Fig. 5.1 shows the effect of tuning the learning rate and the number of hidden dimensions for the HGT regression models used. The graphs show the sweep for each of the four KPIs chosen, with "Orders -> PayOrder" representing the "Elapsed time from Orders to PayOrder" KPI (This nomenclature was used to avoid overloading the graphs with text) and the other three KPIs following the same style.

Beginning with the two graphs shown at the top, referring to the Order Management KPIs, a first observation shows a great difference in volatility between the models' validation error when trained on the 0.01 learning rate versus the lower 0.001 value. The first continues increasing as more hidden dimensions are added, whereas the second is much more stable. It's important to note that the validation error is also higher for the "Elapsed time from Orders to PackageDelivered" KPI as opposed to the values shown for "Elapsed time from Orders to PayOrder"

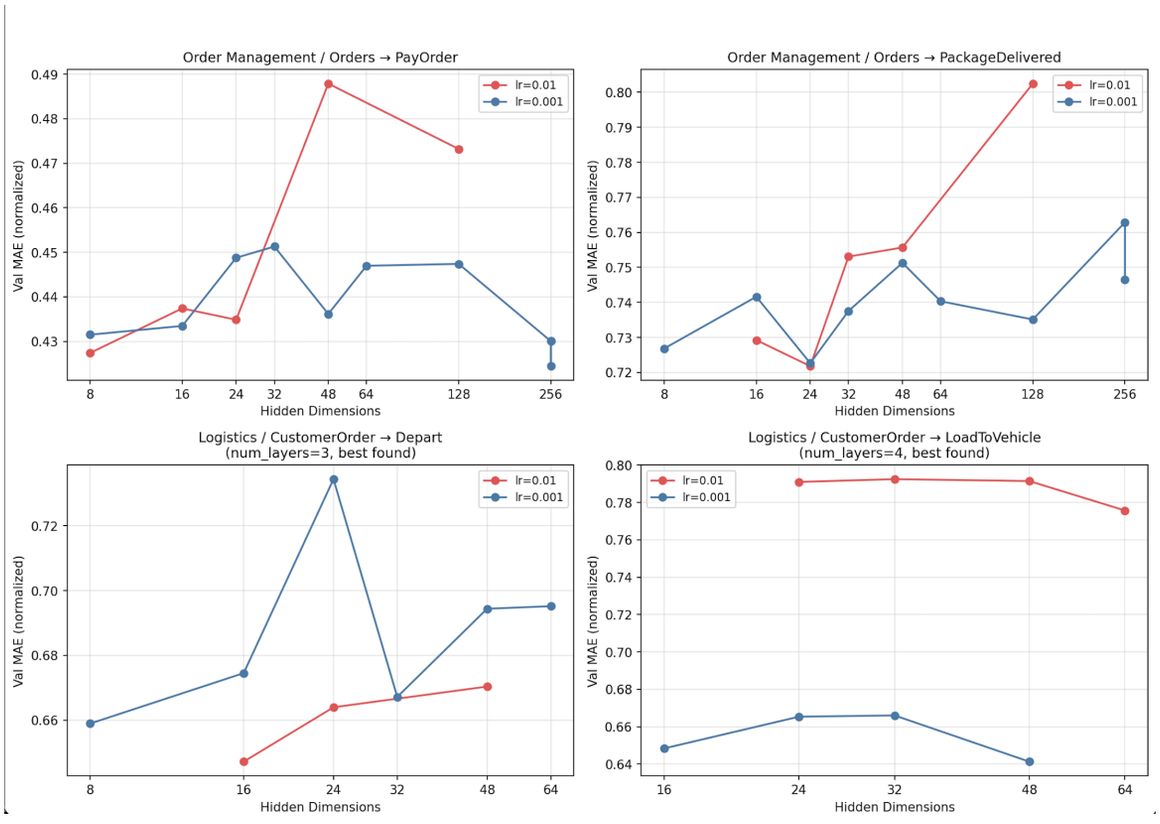


Figure 5.1: Normalized Val MAE scores for the HTG model per hyperparameter setting. Note: scales (y-axis) are not aligned, as I intended to compare hyperparameter settings within each KPI.

KPI. This could be a result of the difference in process execution length for each given KPI. A notable difference between each KPI model is the chosen hidden dimension value for their respective models, with the "Elapsed time from Orders to PackageDelivered" KPI showing better results with 24 hidden dimensions whereas the "Elapsed time from Orders to PayOrder" KPI showed the best results with the maximum 256 hidden dimensions.

In order to perform the same sweep on the Logistics database it was first necessary to identify the proper number of layers needed to avoid the problem outlined in Section 4.1.2.

Fig. 5.2 and Fig. 5.3 show the result of the initial sweeps performed over all possible hidden channels and learning rates for each KPI in the Logistics dataset. In Fig. 5.2 it is evident that 3 layers offers the lowest overall validation error and the resulting sweep selected this value as the best hyperparameter. On the other hand Fig. 5.3 shows a different case, while the ablation analysis shows the number of layers at 3 having the best results, with its median at 0.683 versus the 0.721 of 4 layers, the complete sweep eventually selected 4 layers as the best option.

The second set of graphs in Fig. 5.1 show the result of the sweep performed on the Logistics KPIs once the number of layers were set at the appropriate value of 3 and 4 respectively. It is important to note the different y-axis scales used by each graph as they are only intended for comparing settings within the model. Looking at the graph it's evident that, in contrast to Order Management's results, the Logistics-trained models don't have a generally agreed-upon set of hyperparameter characteristics. The "Elapsed time from CustomerOrder to Depart"

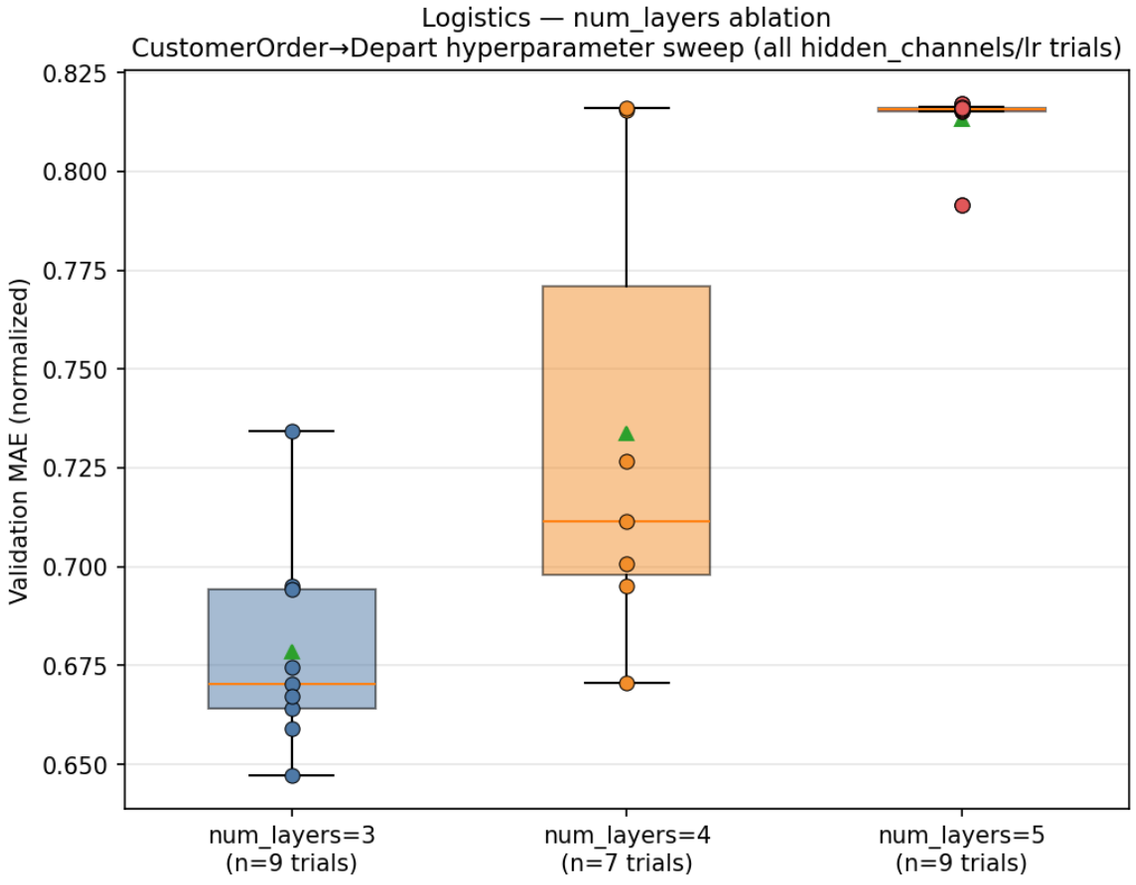


Figure 5.2: Normalized Val MAE scores for the HGT model on the Logistics dataset for KPI "Elapsed time from CustomerOrder to Depart" per different number of layers

KPI shows considerably better results when using a higher learning rate and a smaller number of hidden dimensions than either of the Order Management models. However, the sweep for "Elapsed time from CustomerOrder to LoadToVehicle" KPI shows a much flatter curve where the best values are provided by the lower learning rate and a hidden dimension value of 48.

5.1.2 Model comparison versus homogeneous GNN

Fig. 5.4 shows the learning curves for the best performing regression model on the Order Management dataset, which is associated with the prediction of the "Elapsed time from Orders to PayOrder" KPI. As can be seen in the curves, even after performing a hyperparameter sweep the model cannot be improved below a 0.45 MAE. Notably, training loss stays above validation loss for the entire run rather than the reverse, which at first glance suggests overfitting; however, re-evaluating the fully-trained checkpoint directly on each split (rather than reading the pattern off the epoch-by-epoch curve) shows train MAE (388.0 hours) is genuinely higher than validation MAE (353.2 hours), with test MAE the worst of the three (421.2 hours). I then compared each split's target distribution di-

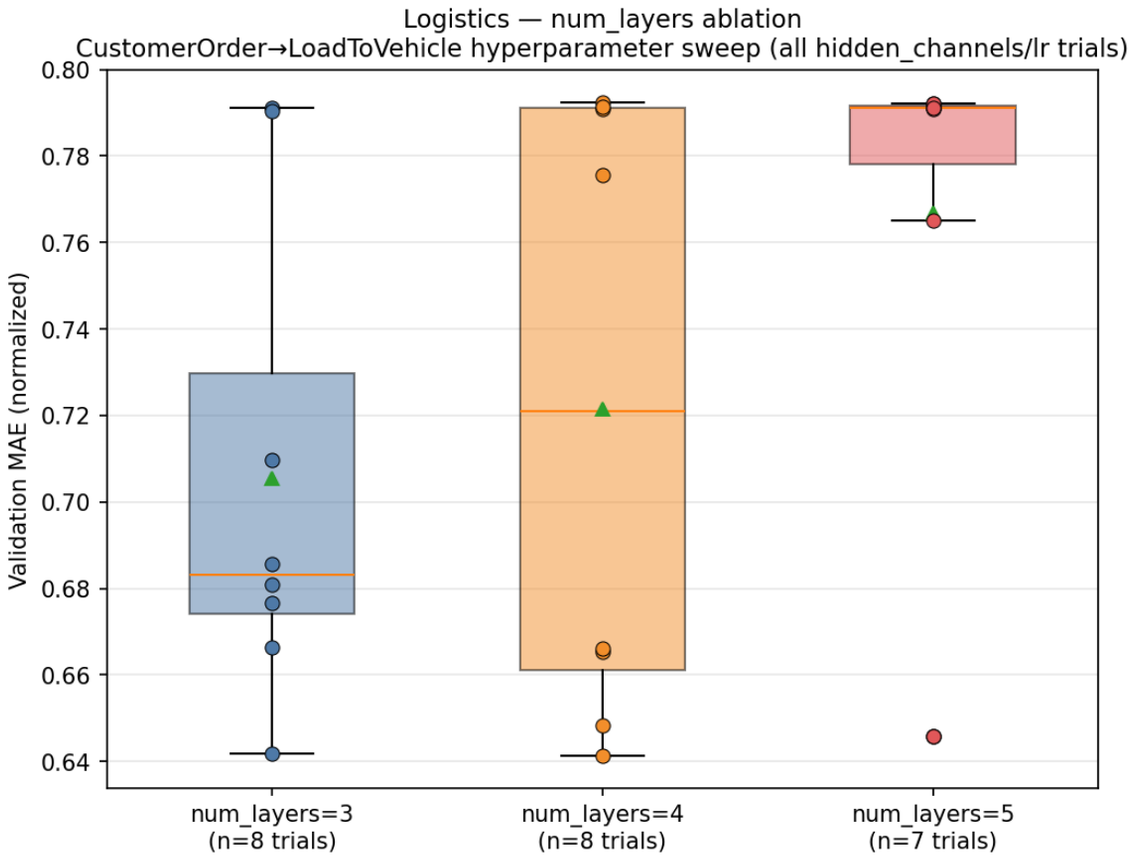


Figure 5.3: Normalized Val MAE scores for the HGT model on the Logistics dataset for KPI "Elapsed time from CustomerOrder to LoadToVehicle" per different number of layers

rectly, independent of any trained model, to validate whether this behavior was tied to the data source itself. I found the validation window's remaining-time values to be intrinsically less variable than the training window's (standard deviation of 135.8 hours versus 219.6 hours), so even a constant mean-predictor would score better on validation (105.7 hours MAE) than on training (158.1 hours MAE) suggesting temporal drift in the dataset. Test's higher error similarly reflects genuine temporal drift toward longer remaining times later in the dataset's timeline, not a generalization failure. Two candidate explanations for the pattern were checked directly and ruled out. The first was the possibility that the gap was caused by the z-normalization statistics being fit only on the training split. However, since the denormalized MAE metric already corrects for this uniformly across splits this explanation was discarded. Secondly, this pattern could be caused by too few training epochs. This explanation was also discarded since the gap traces to an intrinsic property of each split's data rather than an undertrained model. Training was still halted once the early-stop trigger activated after validation MAE stopped improving. The homogeneous implementation, which is allowed to train for longer, shows a similar overall pattern, with a decrease in training error not producing a decrease in validation error, instead increasing again after finding its local minimum at epoch 4 until it's also cut off by the early-stop trigger. Its smoother-looking curve does not translate into a clear performance advantage over the heterogeneous model for this KPI. HomoGNN's all-prefix

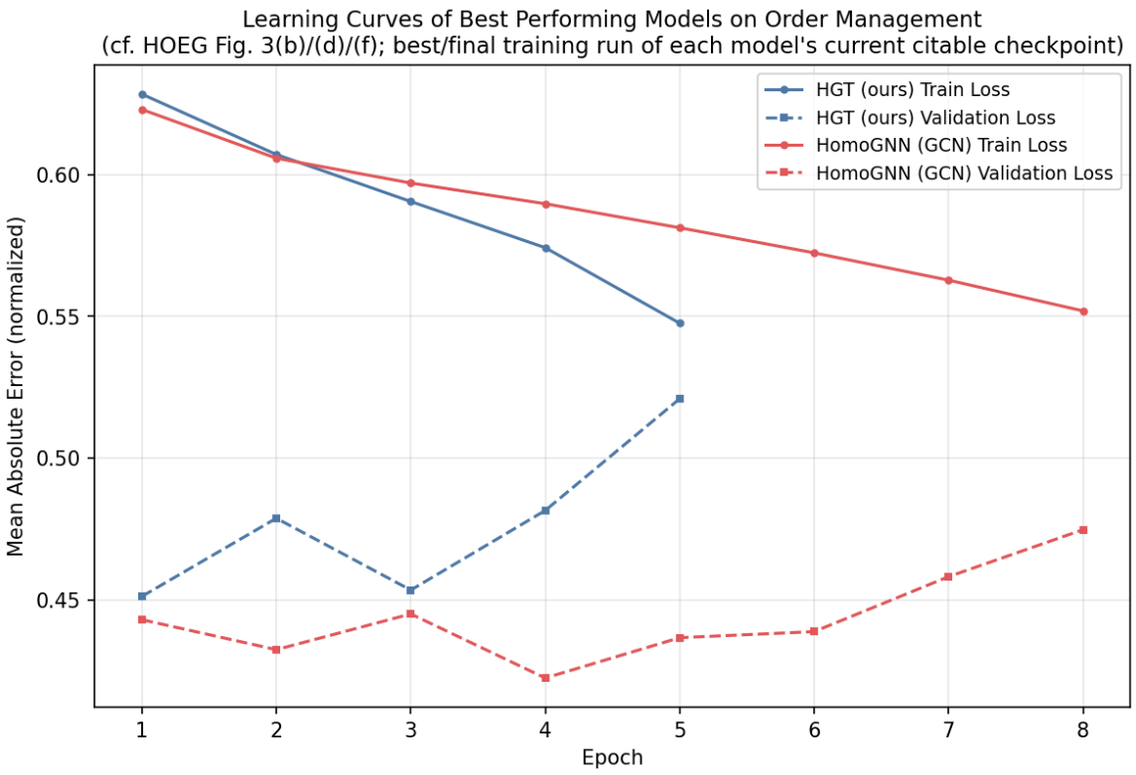


Figure 5.4: Training and validation loss learning curves of the trained HGT model and the homogeneous GNN on the best performing model of the Order Management dataset.

MAE is slightly lower (157.5 hours versus 167.1 hours), but its last-event MAE remains higher than the heterogeneous model's (103.8 hours versus 98.0 hours). A full baseline comparison is presented later in this chapter that provides a complete picture across both metrics.

Fig. 5.5 shows the learning curves for the best performing regression model on the Logistics dataset, which is associated with the prediction of the "Elapsed time from CustomerOrder to Depart" KPI. Both the homogeneous and heterogeneous models show a genuine overfitting-like divergence toward the end of the plotted run, with validation loss rising as training loss keeps dropping, and neither curve having visibly stabilized by the final epoch shown. This divergence, however, occurs entirely after the epoch that early stopping actually selected. Logistics uses a patience of 10 epochs, so the plotted curve necessarily extends 10 epochs beyond the checkpoint that was saved and is cited elsewhere in this thesis, and it is in those extra, discarded epochs that training loss continues falling while validation loss rises. By reloading the actually-deployed checkpoint and re-evaluating it directly I confirmed that it does not carry this gap. Train, validation and test MAE are all within about 8 hours of each other (432.1h/435.1h/439.9h). Unlike what can be seen in the results for the order management dataset for this dataset the two models track each other closely throughout.

Overall, neither graph reflects overfitting once the pattern behind it is more closely analyzed. Order Management's apparent gap is a property of its temporal validation split rather than a generalization failure, and Logistics' apparent divergence occurs only in training epochs beyond the one actually deployed. Nevertheless, both datasets

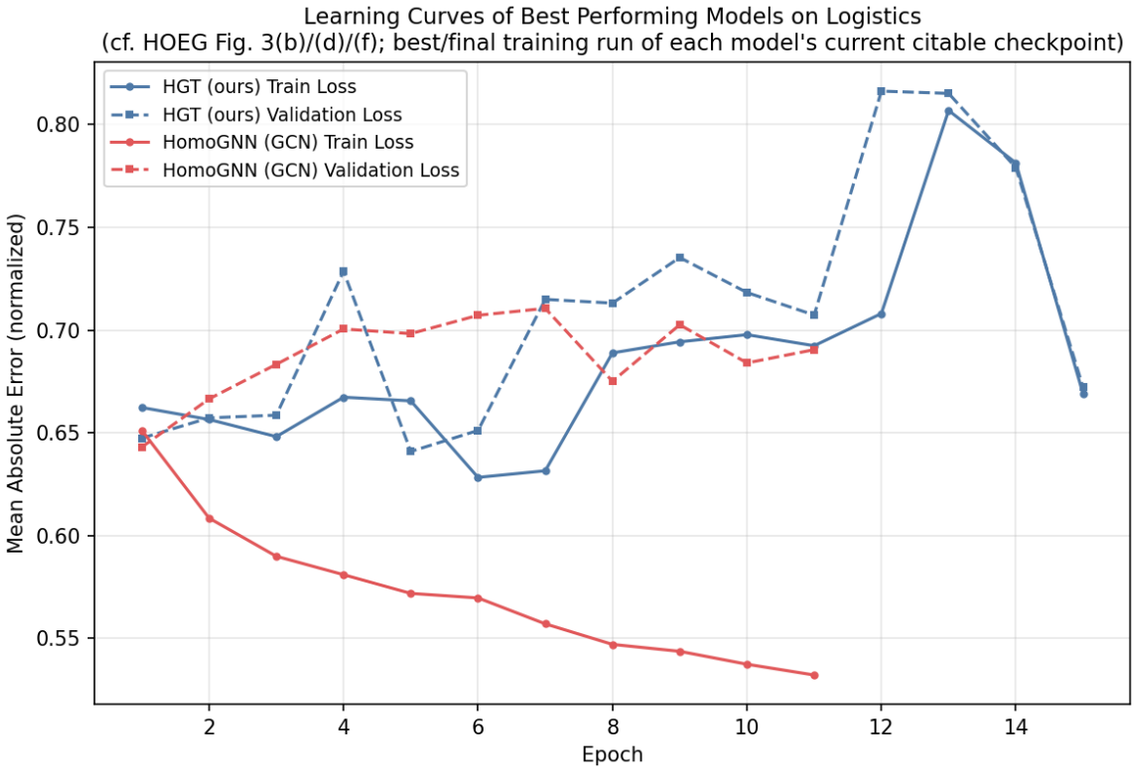


Figure 5.5: Training and validation loss learning curves of the trained HGT model and the homogeneous GNN on the best performing model of the Logistics dataset.

share a training process that plateaus early, with the best available hyperparameters still leaving both model architectures at a relatively high absolute MAE floor for their respective KPIs.

5.1.3 Model comparison against baselines

I compared the heterogeneous model against three other implementations: the mean, a Gradient Boosted Trees model and a homogeneous implementation of the dataset. By comparing the performance to these baseline models I wanted to understand how well the heterogeneous implementation stacked up to other options. Fig. 5.6 shows the results, with the MAE value denormalized and shown as hours. As I explained in Section 4.2.5 I was interested in checking the quality of the model not only on the entire prefix set but also as the prefix depth increased to evaluate if the increase in information was leading to an increase in predictive quality, Therefore, the graph presents the MAE values for both, the model tested on every prefix in the testing set as well as only those marked as Last-Event prefixes, which contain the maximum amount of information possible about the process execution.

As expected, the mean predictor shows the worst overall performance for two of the four KPIs (”Time from CustomerOrder to Depart” and ”Time from CustomerOrder to LoadToVehicle”). For the other two, however, it instead outperforms both the heterogeneous model and GBT. In these two its error is slightly below both on

Baseline comparison — all 5 currently-relevant models

Dataset / KPI	Model	MAE (all) [h] (95% CI)	RMSE (all) [h]	R^2 (all)	MAE (last) [h] (95% CI)	RMSE (last) [h]	R^2 (last)
Order Management / Orders → PayOrder	**HGT**	**167.1 [159.2, 174.5]**	**254.2**	**-0.317**	**98.0 [86.4, 110.5]**	**138.3**	**0.051**
	HomoGNN (GCN)	157.5 [150.1, 164.4]	240.1	-0.174	103.8 [94.1, 115.3]	134.1	0.107
	k-dim GNN (HOEG)	170.4 [162.7, 177.6]	253.9	-0.313	95.8 [86.1, 105.6]	122.7	0.252
	Mean predictor	163.2 [156.2, 169.8]	238.2	-0.155	150.0 [140.1, 160.9]	172.4	-0.476
	GBT	168.8 [161.4, 175.7]	247.5	-0.248	134.4 [123.2, 146.9]	163.5	-0.328
Order Management / Orders → PackageDelivered	**HGT**	**196.8 [189.2, 204.2]**	**272.9**	**-0.517**	**158.4 [141.8, 175.5]**	**203.5**	**-1.056**
	HomoGNN (GCN)	196.0 [188.4, 203.5]	273.6	-0.525	151.4 [135.1, 168.4]	197.4	-0.934
	k-dim GNN (HOEG)	167.5 [159.9, 174.6]	249.7	-0.270	100.7 [90.0, 112.1]	134.4	0.103
	Mean predictor	163.2 [156.2, 169.8]	238.2	-0.155	150.0 [140.1, 160.9]	172.4	-0.476
	GBT	168.8 [161.4, 175.7]	247.5	-0.248	134.4 [123.2, 146.9]	163.5	-0.328
Logistics / CustomerOrder → Depart	**HGT**	**112.0 [110.2, 113.8]**	**143.2**	**0.304**	**6.1 [5.4, 7.0]**	**7.7**	**-0.170**
	HomoGNN (GCN)	112.7 [110.9, 114.6]	144.4	0.292	12.8 [9.8, 16.6]	24.3	-10.598
	k-dim GNN (HOEG)	120.4 [118.8, 122.0]	146.1	0.275	52.3 [43.8, 62.1]	75.7	-111.270
	Mean predictor	141.3 [139.3, 143.4]	172.6	-0.011	301.1 [300.0, 302.4]	301.2	-1774.467
	GBT	125.1 [123.2, 126.8]	153.4	0.202	17.7 [15.4, 20.1]	22.5	-8.945
Logistics / CustomerOrder → LoadToVehicle	**HGT**	**115.0 [113.2, 116.9]**	**143.9**	**0.281**	**8.4 [7.5, 9.3]**	**9.9**	**-30.838**
	HomoGNN (GCN)	123.9 [122.1, 125.9]	153.0	0.187	39.7 [34.0, 45.5]	53.4	-915.651
	k-dim GNN (HOEG)	118.4 [116.5, 120.4]	149.7	0.222	25.4 [19.2, 32.7]	49.4	-785.742
	Mean predictor	140.4 [138.4, 142.5]	170.6	-0.011	304.2 [303.9, 304.5]	304.2	-29796.900
	GBT	126.8 [125.0, 128.7]	154.5	0.170	23.7 [20.0, 27.8]	33.4	-358.135

Figure 5.6: Performance of the chosen HGT model and three baseline models. Note: the best-performing hyperparameter configurations are used for HGT

”Time from Orders to PayOrder,” and it is in fact the single best-performing model of the four on ”Time from Orders to PackageDelivered.” GBT’s own performance is similarly inconsistent across KPIs rather than uniformly ranked. It is the single worst-performing model on PayOrder, the second-best (behind only the mean predictor) on PackageDelivered, and mid-pack on the two Logistics KPIs, beating only the mean predictor on each of those. Comparing the graph-based models it’s clear that neither model can be identified as truly better than the other, with their respective accuracy values being very close. Looking at the confidence intervals of each model also shows that both models tend to have overlapping estimate ranges. Therefore, given the spread between models, it cannot be claimed with certainty that the heterogeneous representation is head-and-shoulders above the others, echoing the results obtained in previous works.

This changes if we compare the models only on the last-event prefix, though not as uniformly as it may first appear. For two of the four KPIs (”Time from CustomerOrder to Depart” and ”Time from CustomerOrder to LoadToVehicle”) the heterogeneous model is significantly more accurate than every baseline (Wilcoxon signed-rank $p < 0.001$ against each of the three baselines and the k-dim GNN comparison introduced in Section 4.2.5). For ”Time from Orders to PayOrder” it is significantly better than the homogeneous GNN, the mean predictor, and GBT, but statistically indistinguishable from the k-dim GNN baseline ($p=0.31$). For ”Time from Orders to PackageDelivered,” the heterogeneous model does not win at all. The heterogeneous model is statistically tied

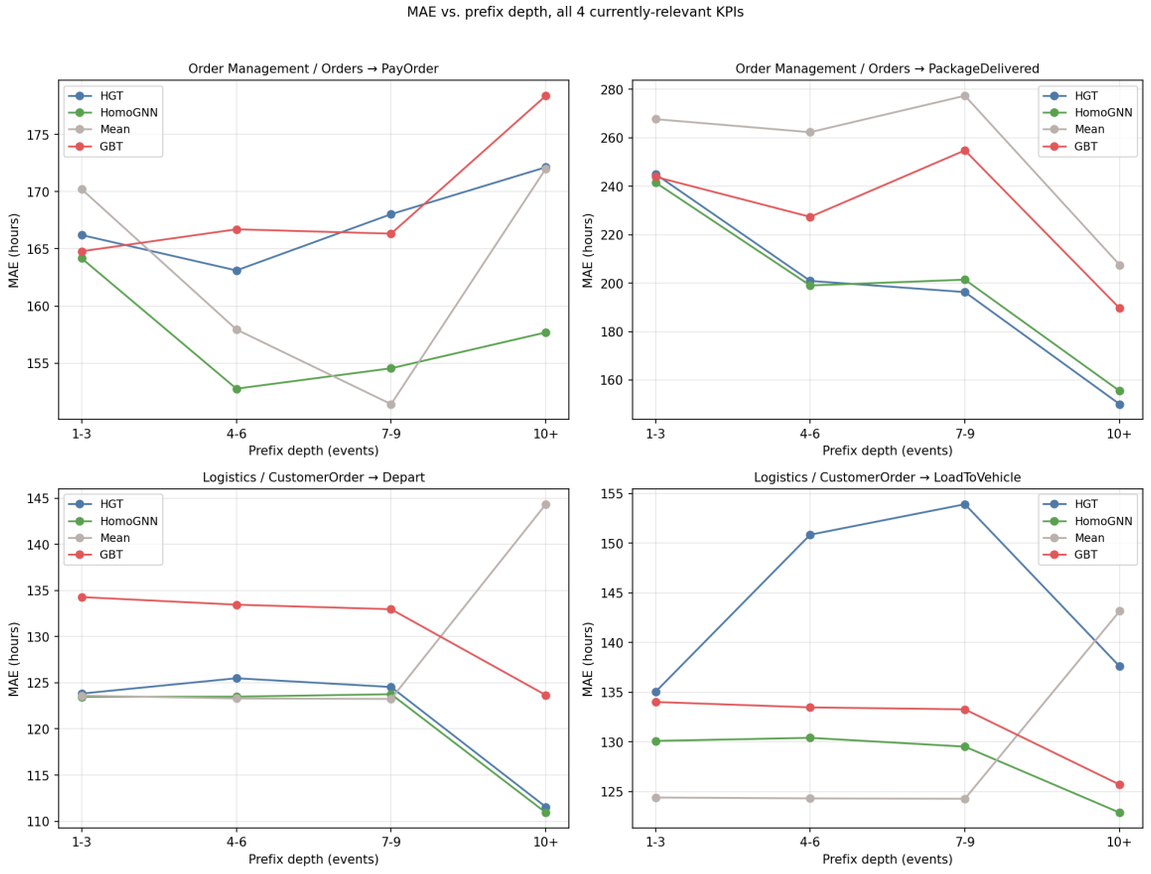


Figure 5.7: Test MAE for each regression model across prefix depth.

with the homogeneous GNN (p=0.48) and significantly less accurate than the mean predictor, GBT, and the k-dim GNN baseline. One further caveat applies on top of this for "Time from CustomerOrder to LoadToVehicle" KPI, which has a strongly negative R^2 for every model, most extremely for the mean predictor (R^2=-29796). This is a consequence of the last-event target's true scale being tiny, with a mean of 2.3 hours and a standard deviation of only 1.8 hours (an order of magnitude smaller than the other Logistics KPI, which has a mean of 27 hours and a standard deviation of 7 hours). Since R^2 is calculated against this tiny variance, even the heterogeneous model's own comparatively modest last-event error produces a negative R^2 (-30.8), smaller in magnitude than the baselines' because its MAE is itself the lowest of the five, but still negative for the same structural reason. Overall, MAE remains the more informative metric for this KPI. As shown above, the heterogeneous model's last-event MAE is significantly lower than every baseline's here, unlike the mixed picture for PayOrder and PackageDelivered.

Fig. 5.7 shows this change in predictive quality over prefix depth. The graphs show that in only two of the four KPIs the model's performance works as expected, with the MAE value decreasing as more information is added to the process execution. The HGT model for the "Time from CustomerOrder to LoadToVehicle" KPI is especially problematic as it's the only model that increases across the prefix depth. These graphs also highlight an important result. The HGT model performs worse for three of the four KPIs, with its MAE always higher than the

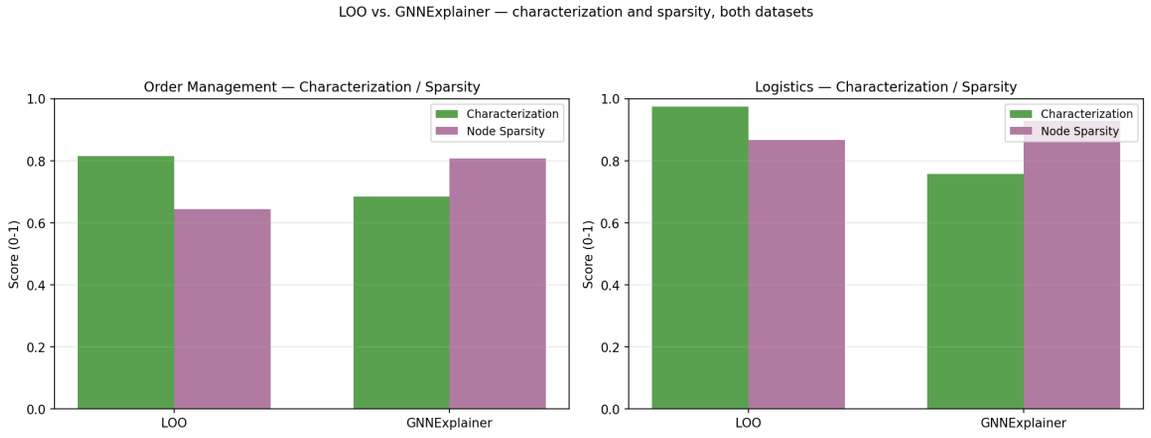


Figure 5.8: Fidelity metrics of LOO method against GNNExplainer for KPIs of each dataset

homogeneous model's across multiple prefix depths. This result reverses for two of the four KPIs when comparing the models exclusively on last-event prefixes, as shown in the statistical breakdown above, indicating that aggregate metrics, which are dominated by non-last-event prefixes, and last-event-specific metrics can move independently, sometimes in opposite directions. However, it is worth noting that the aforementioned breakdown also shows this reversal is itself KPI-dependent rather than universal.

5.2 Explainability framework

Having described the predictive model's results I now show the results of applying my proposed explainability framework on the created models. I show the output of each of the four distinct components of the framework as well as a short explanation of how the provided explanations can be interpreted by a stakeholder.

5.2.1 Perturbation-based explanation

As I explained in Section 3.4.1 I performed a series of tests to validate a change in the way this part of the methodology generated its results. Fig. 5.8 shows the results of this evaluation. I compared the two methods on two explanation fidelity metrics: characterization and sparsity. LOO outperformed GNNExplainer on characterization across both datasets (Order Management LOO 0.815 vs. GNNExplainer 0.685; Logistics LOO 0.975 vs. GNNExplainer 0.757), meaning that features identified through LOO have a larger impact on the model's prediction than those found by the heterogeneous implementation of GNNExplainer. The reverse holds for node sparsity, where GNNExplainer's more selective masks score higher in both datasets (Order Management GNNExplainer 0.807 vs. LOO 0.645; Logistics GNNExplainer 0.929 vs. LOO 0.868). Overall, LOO's exhaustive search identifies a more impactful, but proportionally larger, set of important elements. Logistics' characterization score is, in fact, the higher of the two datasets rather than the lower one, indicating that LOO-identified features are highly decisive for the model's prediction in both settings.

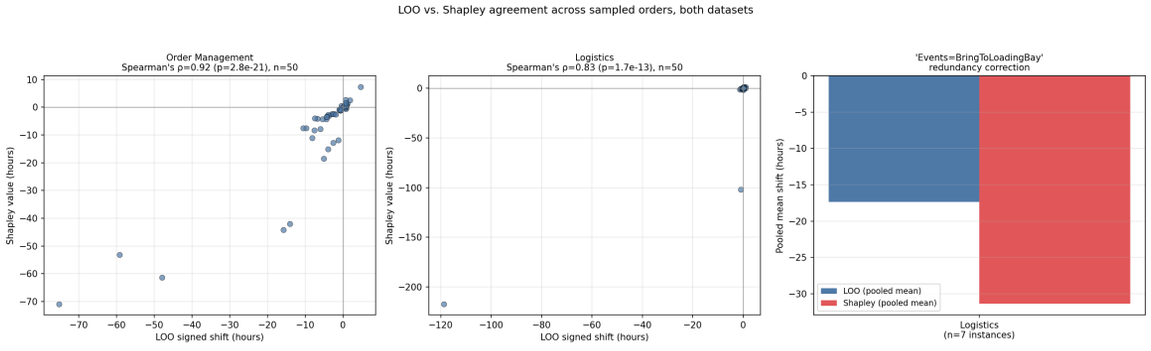


Figure 5.9: Measure of agreement between LOO method and full Shapley Values

Additionally, I needed to validate LOO's effectiveness as a proper identifier to limit the space over which Shapley Values would need to be run. Fig. 5.9 shows the result of this analysis. Both datasets show a strong, statistically significant rank agreement between the values identified by the two methods: Order Management reported a Spearman's ρ value of 0.92 (p=2.8e-21), and Logistics reported a value of 0.83 (p=1.7e-13), each over 50 sampled orders. Despite this strong agreement in ranking, the two methods can still disagree on an element's exact magnitude when its underlying occurrences are redundant with one another. Logistics' `BringToLoadingBay` event, which recurs multiple times within a single process execution, illustrates this. In this case LOO's pooled mean shift across the 7 instances where it appears is -17.3 hours, while Shapley's coalition-based accounting attributes -31.3 hours to the same event, correcting for the redundancy between its repeated occurrences that LOO's one-at-a-time removal underestimates. This highlights the continued need for Shapley Values as a proper quantifying method even where LOO's ranking alone is a reliable and much cheaper identifier.

Before working through these examples, it is worth being explicit about how the single trace shown in them was chosen. Every single-trace figure in this section, as well as the gradient-based and counterfactual explanations shown later in this chapter, reuses the same order (order 1801, Order Management dataset, Orders → PayOrder) as one running example, rather than a fresh trace per figure. This is a deliberate choice made for narrative continuity, not a claim that the trace is representative: order 1801 was originally selected via a "slow trace" heuristic (a trace from the bottom quintile by remaining time, chosen to make for an interesting worked example), with no accuracy or fidelity criterion involved in its selection, and its own explanation fidelity later in this chapter turns out to be below the dataset average rather than favorably cherry-picked (Fig. 5.29, characterization 0.565 against an aggregate LOO characterization of 0.815, Fig. 5.8). The dataset-wide, representative picture instead comes from the holistic aggregate view at the end of this section (Figs. 5.15-5.18), which pools hundreds of predictions rather than illustrating a single one.

Fig. 5.10 shows examples of the graphs shown to the stakeholder for the single trace perturbation-based explanation. As can be seen the provided graphs show the node type and its associated OCEL identifier. The node's impact on the model's prediction is shown in terms of hours, a proper understandable magnitude the stakeholder can interpret, with bar size and color showing the magnitude of the impact and its sign respectively. All these elements are considered in order to make the presented explanation as easily interpretable as possible by a stakeholder with limited knowledge about the workings of the predictive model. Additionally, the stakeholder is also provided with a table that gives a more structured view of the same information as the graphs. The presented

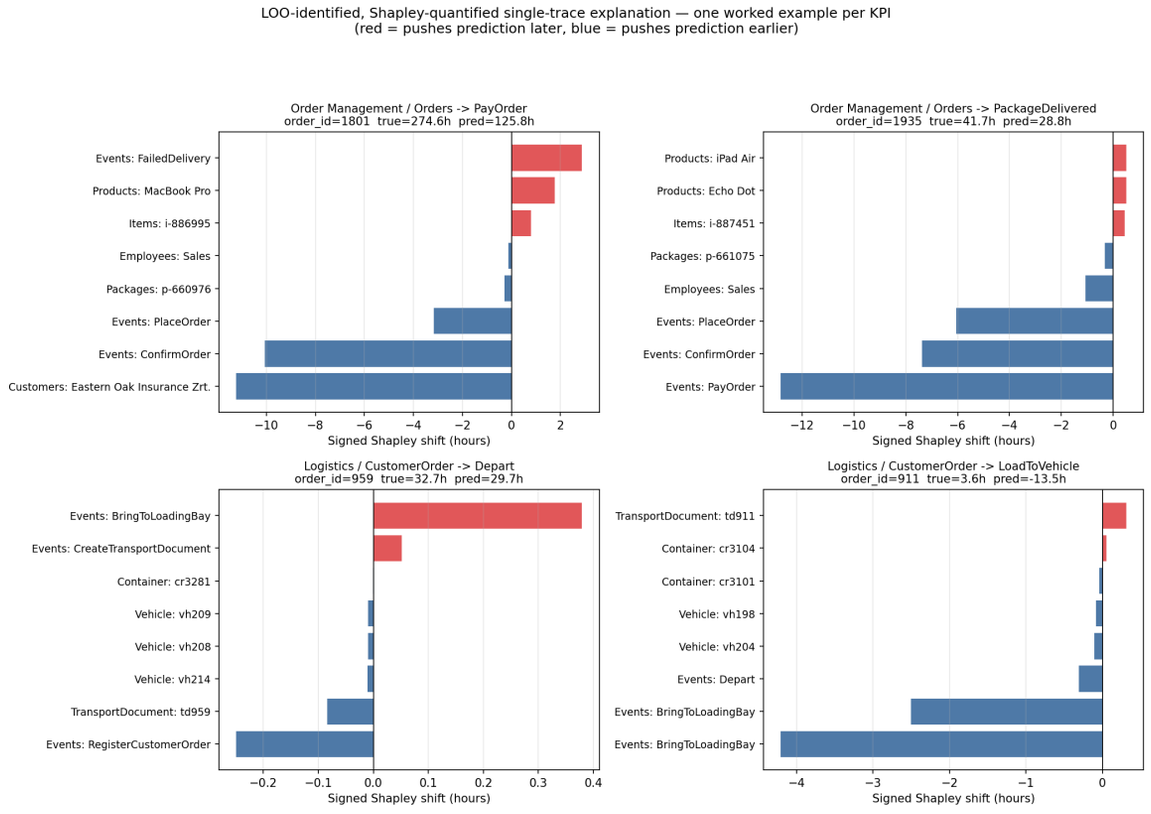


Figure 5.10: Graphs showing the single-trace explanations generated indicating the most important nodes in each trace as well as the impact on the model's prediction.

analysis is applied to the three components of the input graph (nodes, edges and their associated feature values), each graph and table providing a summary of the information for each of these elements. Fig. 5.11 shows the same explanation process applied to the edges of prefixes in each dataset.

Additionally, the end user is also provided with a visualization of the explanation subgraph created by the explanation. As can be seen in Fig. 5.12 the subgraph is composed only of the most important elements identified by the explanation and provides another visual way of understanding the most impactful components of the process execution the model looked at when performing its prediction. In other words, the provided subgraph acts as a visual summary of the most impactful components of the input graph.

Fig. 5.13 shows that the given explanation can be applied to any prefix in a chosen process execution, giving the stakeholder the ability to view how the importance of elements shifts as the process moves along and more information is provided to the model. As can be seen in the figure, the ranking of important elements changes between the early prefix and the last-event prefix for every KPI shown. For example, on the Order Management / Orders → PayOrder KPI, the appearance of the `ConfirmOrder` event is the dominant positive driver on the early prefix but becomes a negative driver by the last event, highlighting a shift in the model's understanding of the data provided as more information becomes available.

Aside from single traces the stakeholder is also provided with an explanation over an aggregate set of traces.

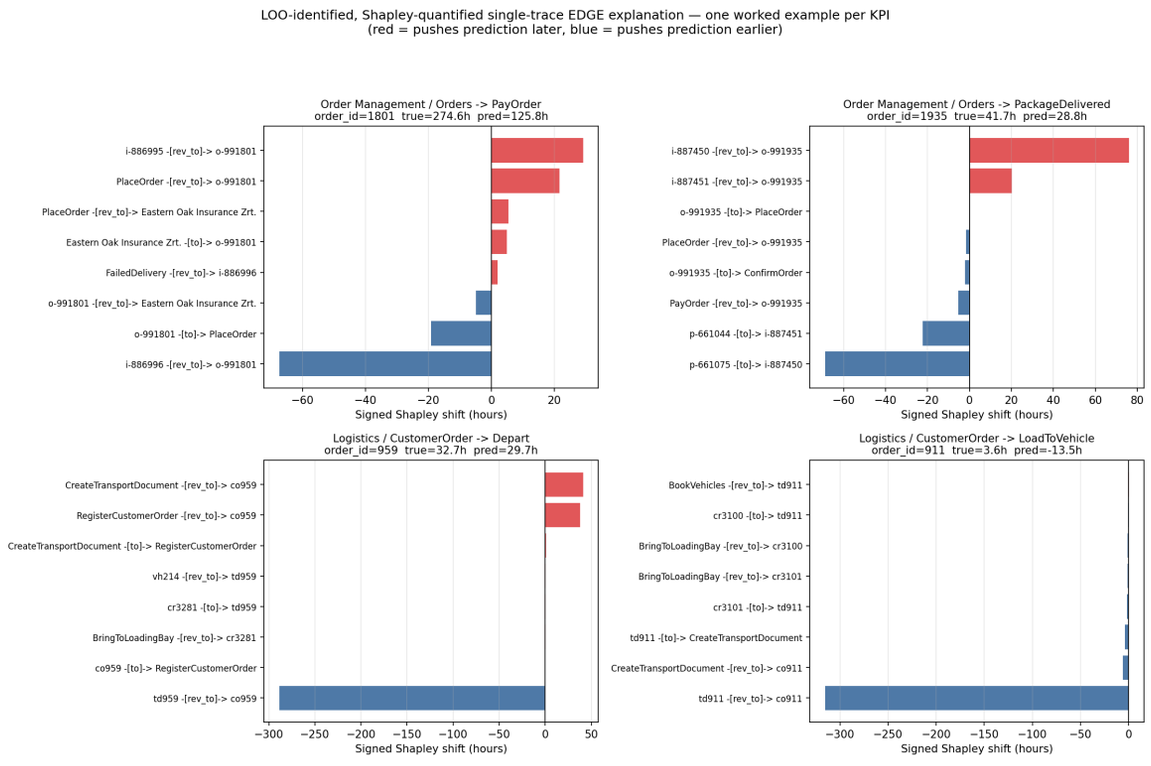


Figure 5.11: Graphs showing the single-trace explanations generated indicating the most important edges in each trace as well as the impact on the model's prediction.

Fig. 5.14 shows an example of the graphs provided measuring the impact of different node types on the model's prediction. The graphs give a rundown of the most important features of a given type (nodes, edges and their associated feature values) over the chosen universe of prefixes. In order to maintain cohesion within the presentation, each graph is designed to mirror its single trace counterpart.

Holistic aggregate view

The graphs shown so far in this section describe the quality of the explanations, i.e. how faithful they are (Fig. 5.8), how consistent LOO is with full Shapley Values (Fig. 5.9) or, in Fig. 5.14's case, show a compact per-KPI summary limited to a handful of elements. However, none of them give a single, holistic view of what the framework identifies as important across the dataset as a whole. Figs. 5.15-5.18 close that gap: each is a single ranked list, pooled across the same 50 traces per KPI used elsewhere in this section, showing every node identity (event type, customer, item, package, transport document, etc.) whose presence shifted the model's prediction, signed and ranked by its average impact. Rather than illustrating one trace or comparing two methods, these graphs give the stakeholder a genuine overview of the model's behavior as a whole by highlighting which recurring elements of the process consistently push predictions earlier or later, independent of any single example.

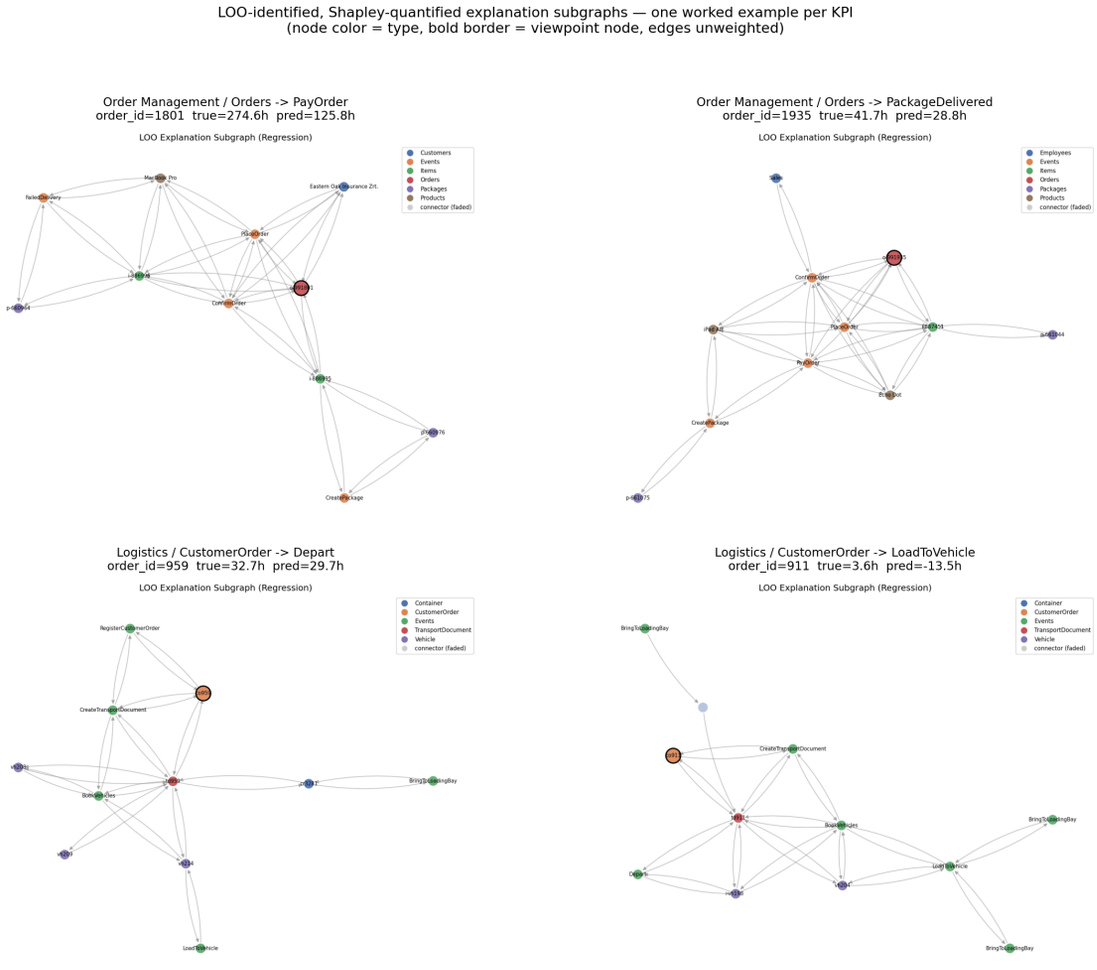


Figure 5.12: Explanation subgraphs created using the identified most important nodes and edges, one worked example per KPI.

5.2.2 Gradient/feature-based explanation

While node and edge importance are calculated exclusively through the use of the perturbation-based method their associated feature values follow a different methodology. Fig. 5.19 and Fig. 5.20 show the ranking of feature importance for the feature values present in a given trace for the Orders node. The values presented in the right side graph of Fig. 5.19 refer to the attribution values calculated through the chosen gradient-based explanation method, in this case Gradient · Input. In order to present this value in a quantifiable way the stakeholder is also shown a graph such as the one in the left side. This graph provides the physical magnitude of the impact for each of the identified feature values from the first analysis. The graph follows the same design as the graphs shown for the other elements of the input graph in order to maintain cohesion in the presentation. Fig. 5.20 showcases that the methodology allows the stakeholder to choose their desired feature attribution method if they wished to compare the rankings provided by each one.

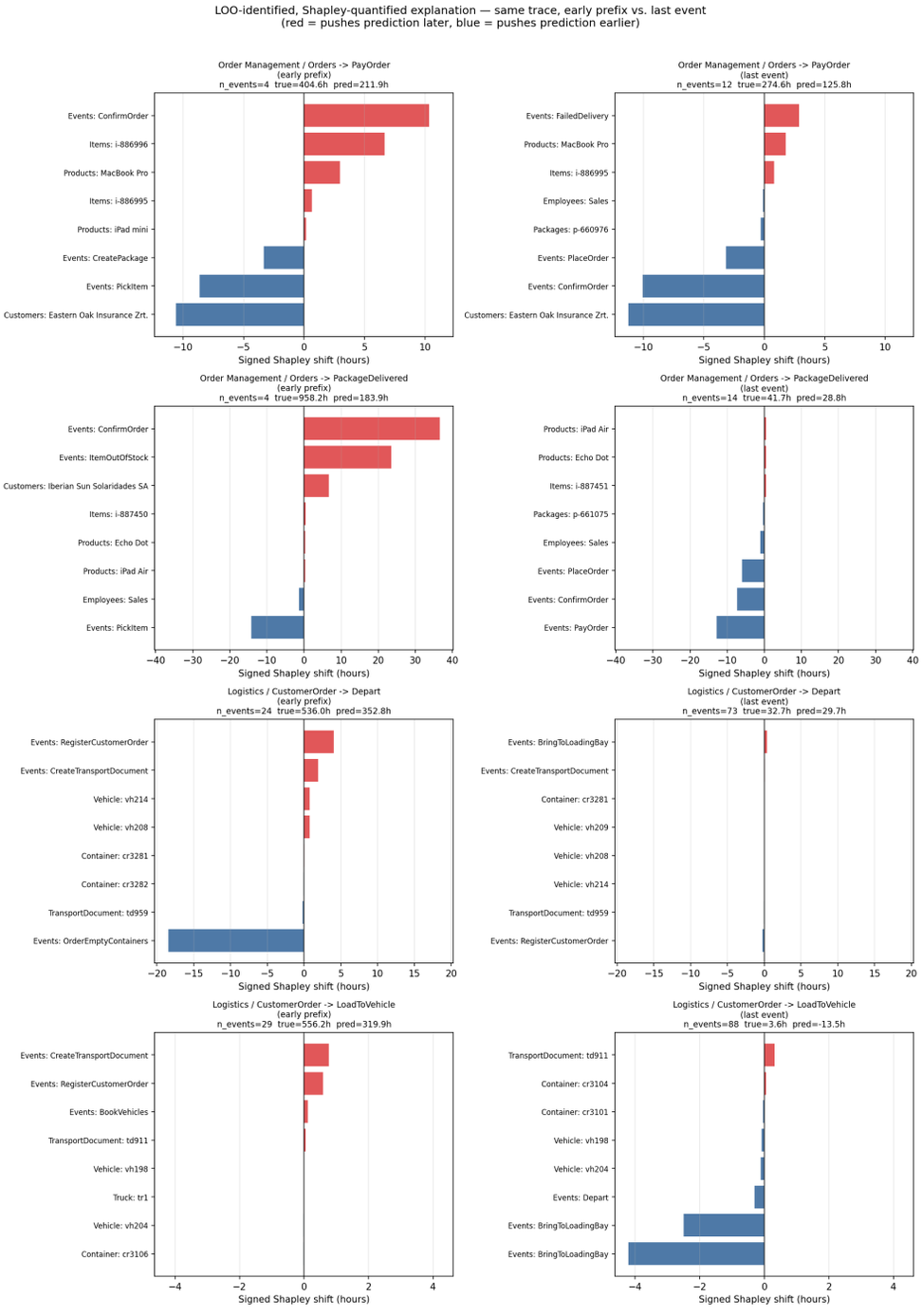


Figure 5.13: Graphs showing the single-trace explanations of node importance generated for an early prefix and the last-event prefix of the same process execution, for each of the four KPIs.

In the same vein as the perturbation-based explanation, this method can also be applied to a collection of prefixes in an aggregate analysis. Fig. 5.21 and Fig. 5.22 show the resulting graphs. Following the pattern shown for

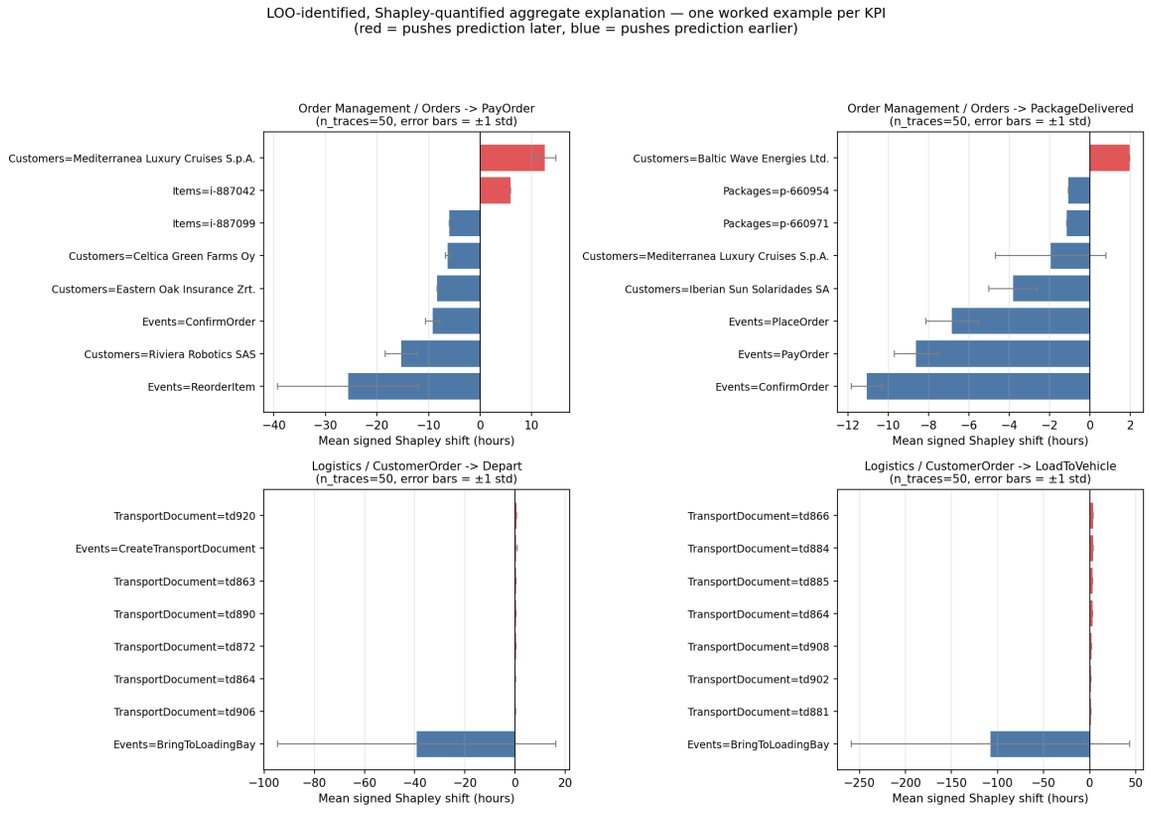


Figure 5.14: Graphs showing the aggregate explanations generated indicating the most important nodes for a collection of 50 traces as well as the impact on the model's prediction.

the first explanation, the stakeholder is presented with a graph that identifies the most impactful features of the nodes over the entire prefix set as well as the average impact their presence has on the model's predicted output. Same as with the single-trace analysis, the stakeholder is able to choose which gradient method they prefer to use for the explanation, allowing them to compare the results of each methodology.

5.2.3 Counterfactual explanation

My proposed counterfactual explanation relies on selecting a graph (graph 1), shown in Fig. 5.23, that the stakeholder wishes to obtain an explanation for and then finding the most similar graph (graph 2) in the existing set that satisfies a given condition. This condition depends on the chosen KPI, in the case of this thesis the condition is that graph 2's remaining time prediction is lower than the remaining time predicted for the graph 1. Fig. 5.25 shows the result of this method. The dissimilarity score for the counterfactual subgraph against the input is shown to the stakeholder broken down between 4 bar graphs, each showing a different value of the total dissimilarity value. The goal of this graph is for the end user to easily identify the graph element that is driving the difference between the two similar graphs. Fig. 5.25 shows an example where the dissimilarity is mainly driven by the difference in features between the two subgraphs.

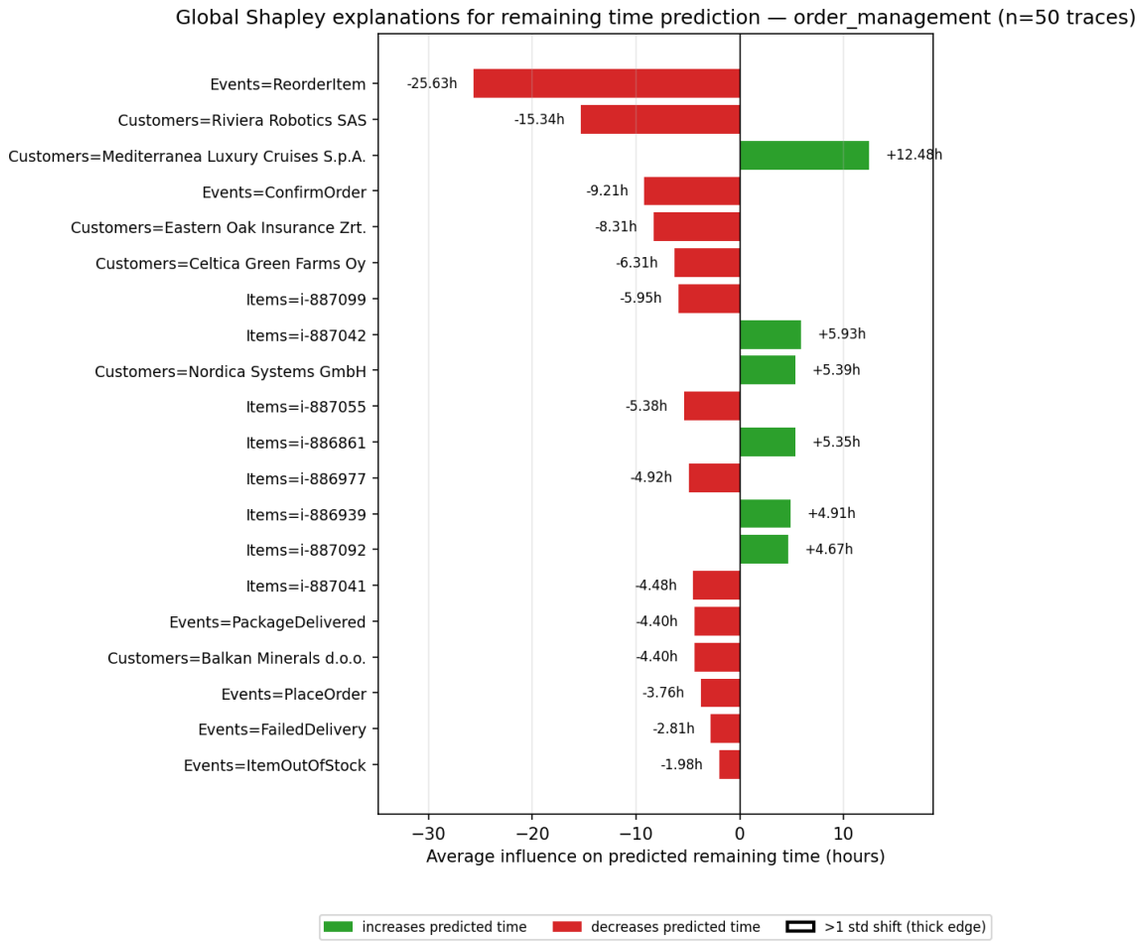


Figure 5.15: Global LOO+Shapley explanation, ranked across all identified elements, for Order Management / Orders → PayOrder (50 traces).

The stakeholder can define the minimum threshold value for the difference in prediction value between the subgraphs. In this example no minimum threshold was enforced (the default value of 0 hours was used), so the candidate set simply consists of the most similar graphs among those with a lower predicted remaining time. As can be seen in Fig. 5.24, the identified counterfactual graph happens to have a total predicted time well below the query graph's (31.4 hours versus the query graph's 125.8) while also maintaining a similar overall structure. As was explained in Section 4.3.3 the chosen counterfactual subgraph is chosen from among a set of candidates identified as having a similar structure to the input graph. Fig. 5.26 shows a table that is also provided to the stakeholder which shows the other two most similar graphs present in the candidates set and their dissimilarity values.

In order to provide the stakeholder with a more actionable way of interpreting this explanation the different elements evaluated for the dissimilarity score are also shown in their own dedicated tables, highlighting the differences between them in the two subgraphs. For example, Fig. 5.27 and Fig. 5.28 show the difference in feature values between the Orders node and the Event nodes of the subgraphs respectively. Given that the dissimilarity

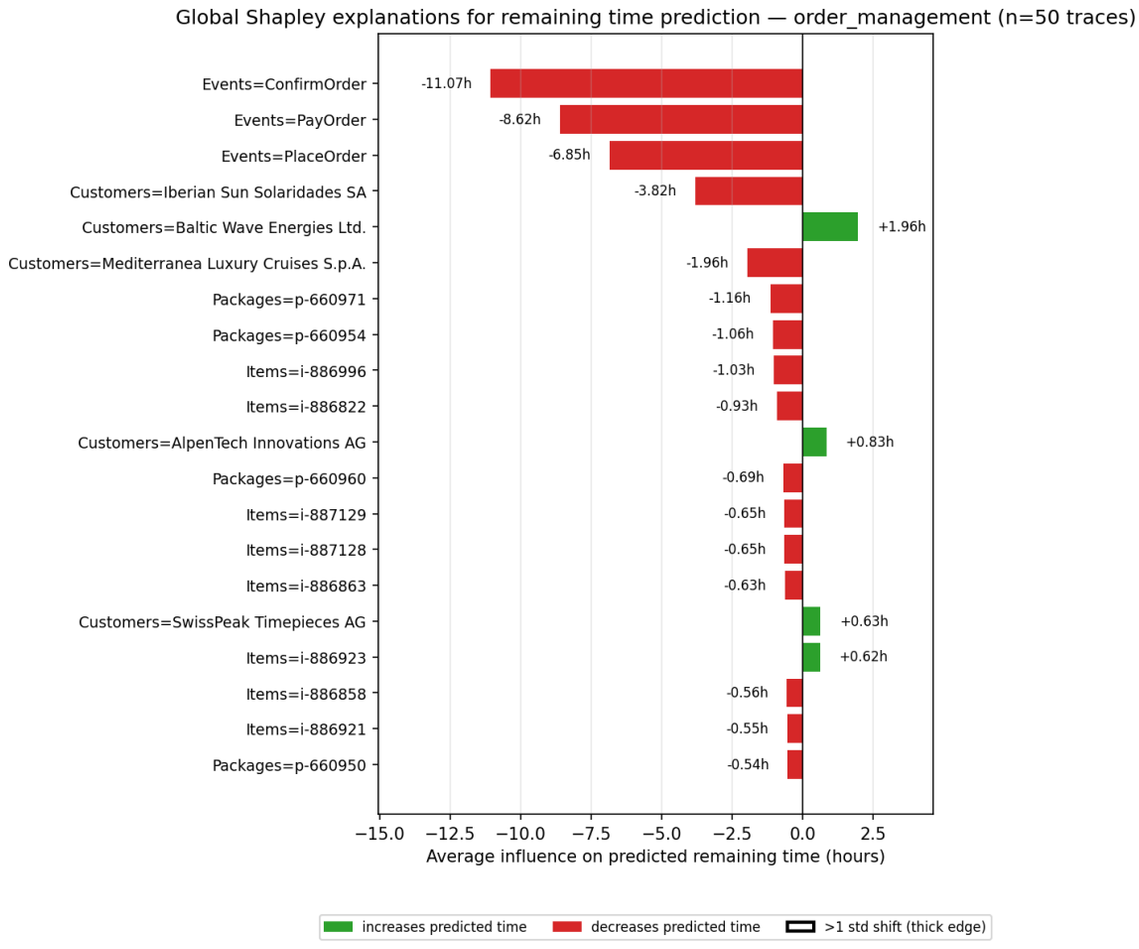


Figure 5.16: Global LOO+Shapley explanation, ranked across all identified elements, for Order Management / Orders → PackageDelivered (50 traces).

was mainly driven by differences in features those different features are identified in these tables and shown to the stakeholder so they may better understand what's driving the model's predictions.

5.2.4 Explanation metrics

Finally, the stakeholder is also provided with fidelity metrics meant to help them assess the faithfulness of the explanation provided. An explanation is generated for the full graph using the framework and then the model's prediction is checked on a subgraph which does not contain the explanation subgraph (fidelity +) followed by a prediction performed on only the explanation subgraph (fidelity -). Fig. 5.29 shows the resulting baseline, fidelity+, and fidelity- graphs for one worked example, continuing the same order 1801 running example introduced earlier in this chapter. The stakeholder is shown only the resulting values for the given metrics, along with a short interpretation of the metric to aid them in their interpretation. Fig. 5.30 shows the table of metrics. In this example the proposed explanation has a characterization score of 0.565, below the 0.815 aggregate LOO characterization re-

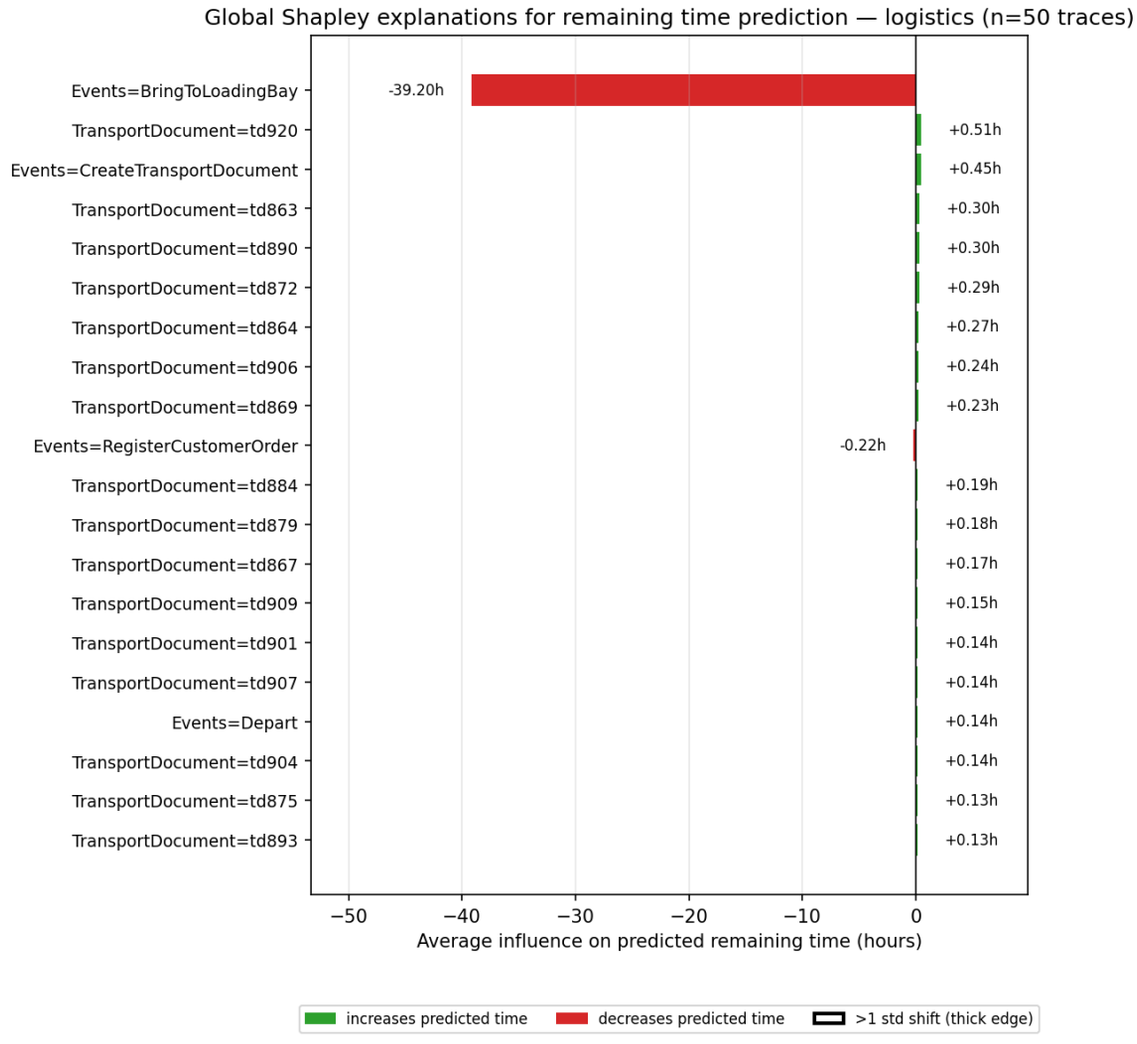


Figure 5.17: Global LOO+Shapley explanation, ranked across all identified elements, for Logistics / CustomerOrder → Depart (50 traces).

ported for Order Management (Fig. 5.8), indicating that, while the proposed explanation is able to identify some of the input's most impactful elements, this particular trace is a comparatively less faithful case rather than the typical one. Its node sparsity value of 64.5% and edge sparsity of 97.6% indicate that the explanation has disregarded more than half of the existing nodes and almost all edges as unimportant to the model's prediction providing a very compact explanation subgraph.

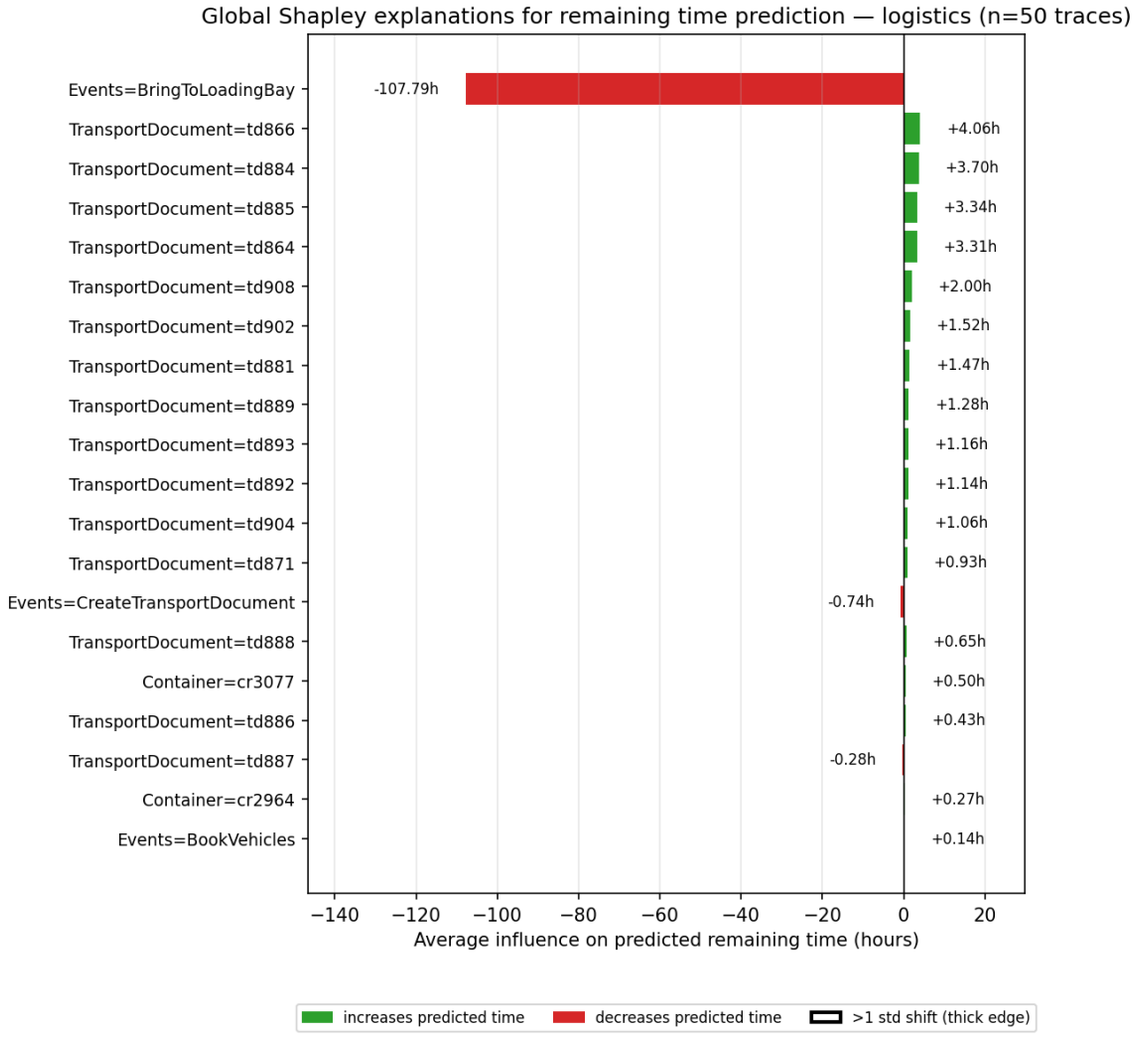


Figure 5.18: Global LOO+Shapley explanation, ranked across all identified elements, for Logistics / CustomerOrder → LoadToVehicle (50 traces).

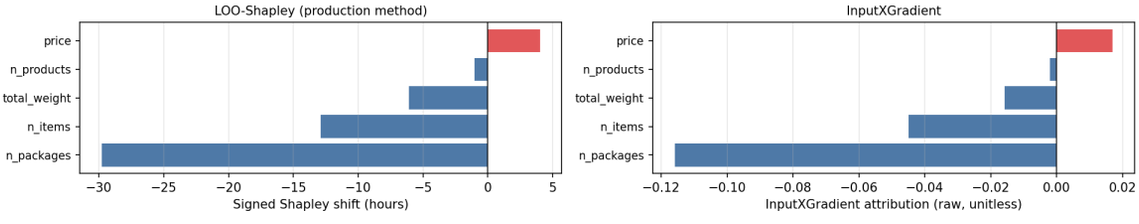


Figure 5.19: Graphs showing the single-trace explanation provided for feature importance on the Order node using the Gradient · Input method for identification and the LOO-Shapley method for quantifying the feature's impact on the prediction.

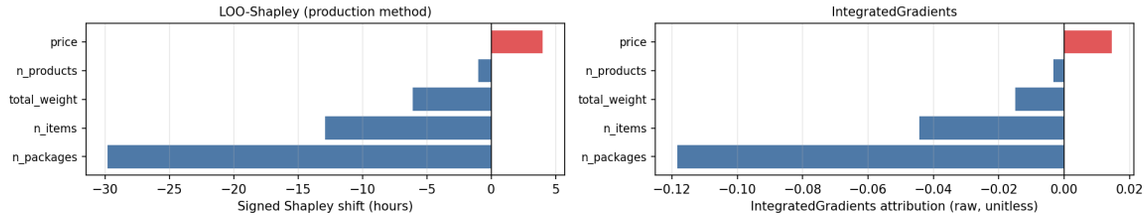


Figure 5.20: Graphs showing the single-trace explanation provided for feature importance on the Order node using the Integrated Gradients method for identification and the LOO-Shapley method for quantifying the feature's impact on the prediction.

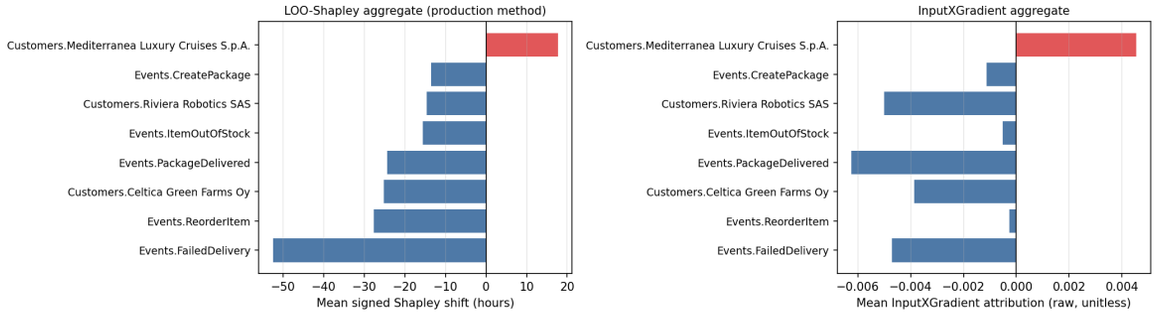


Figure 5.21: Graphs showing the aggregate explanation provided for feature importance of the nodes in the chosen traces using the Gradients · Input method for identification and the LOO-Shapley method for quantifying the feature's impact on the prediction.

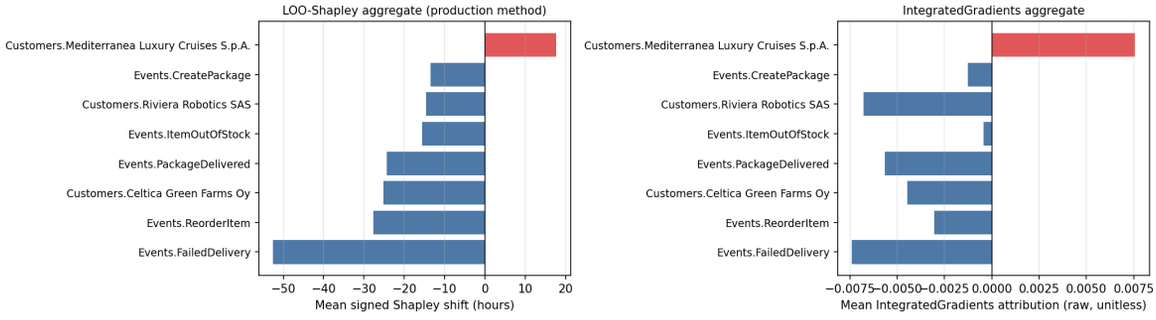


Figure 5.22: Graphs showing the aggregate explanation provided for feature importance of the nodes in the chosen traces using the Integrated Gradients method for identification and the LOO-Shapley method for quantifying the feature's impact on the prediction.

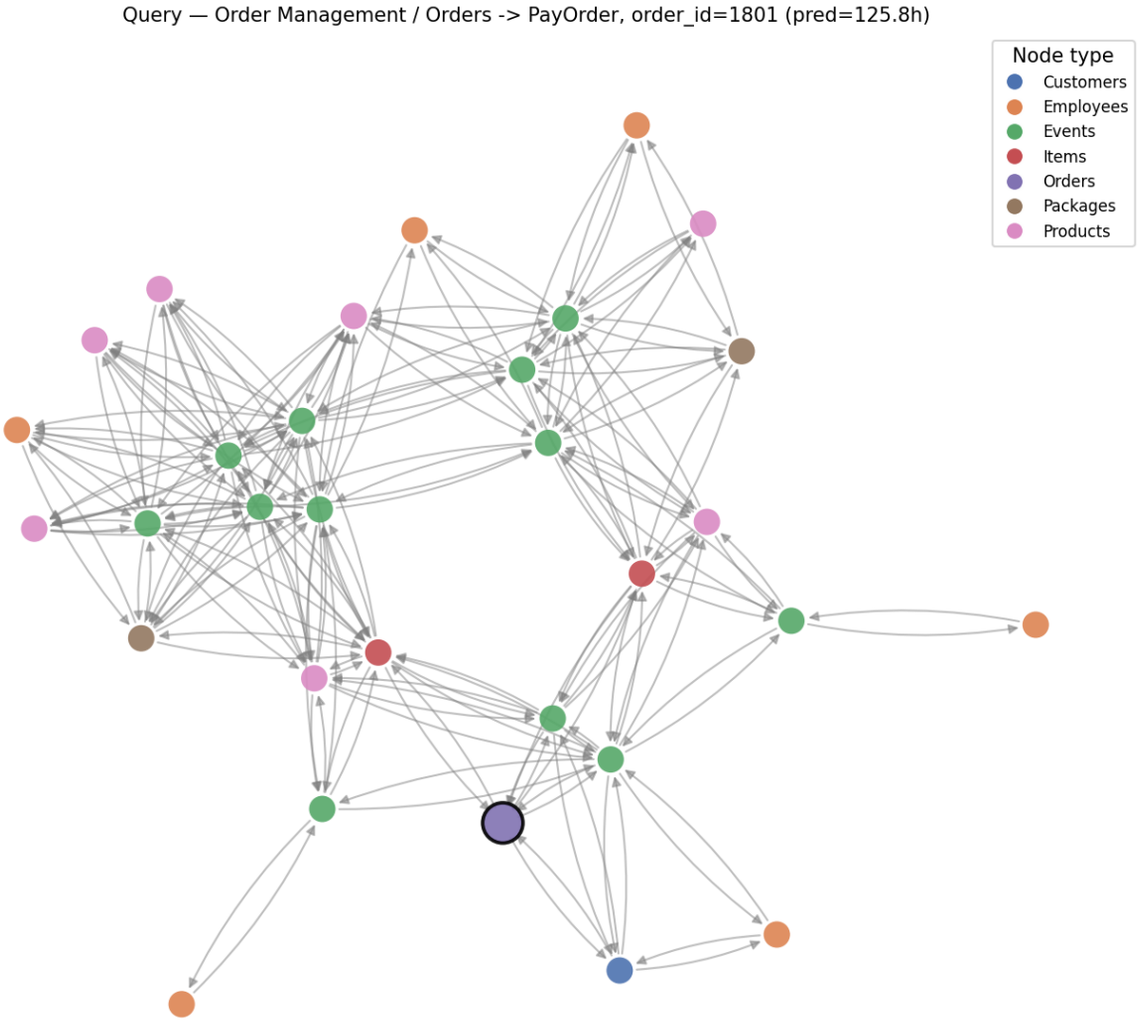


Figure 5.23: Graph representation of the user's selected graph for counterfactual explanation.

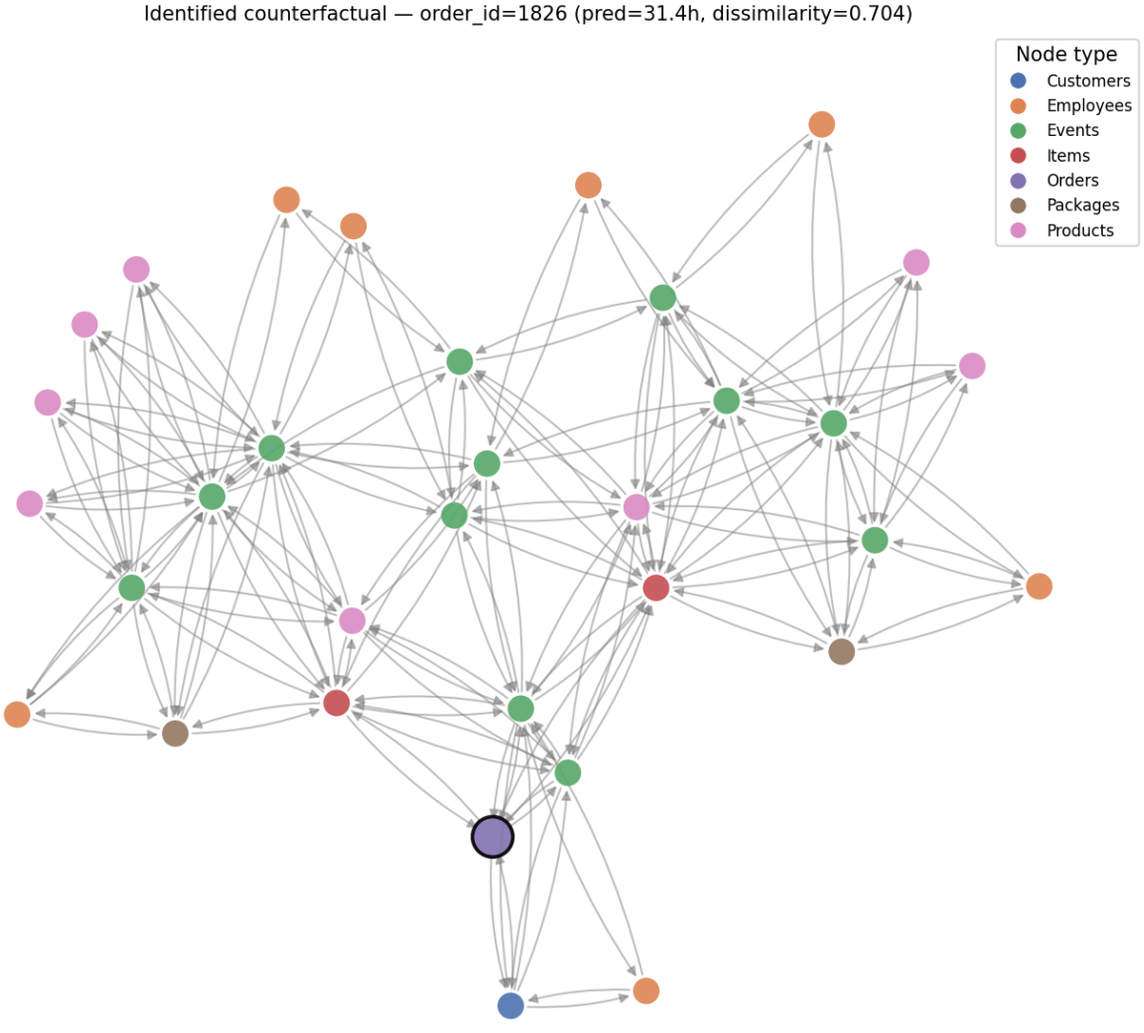


Figure 5.24: Graph representation of the counterfactual graph identified for counterfactual explanation.

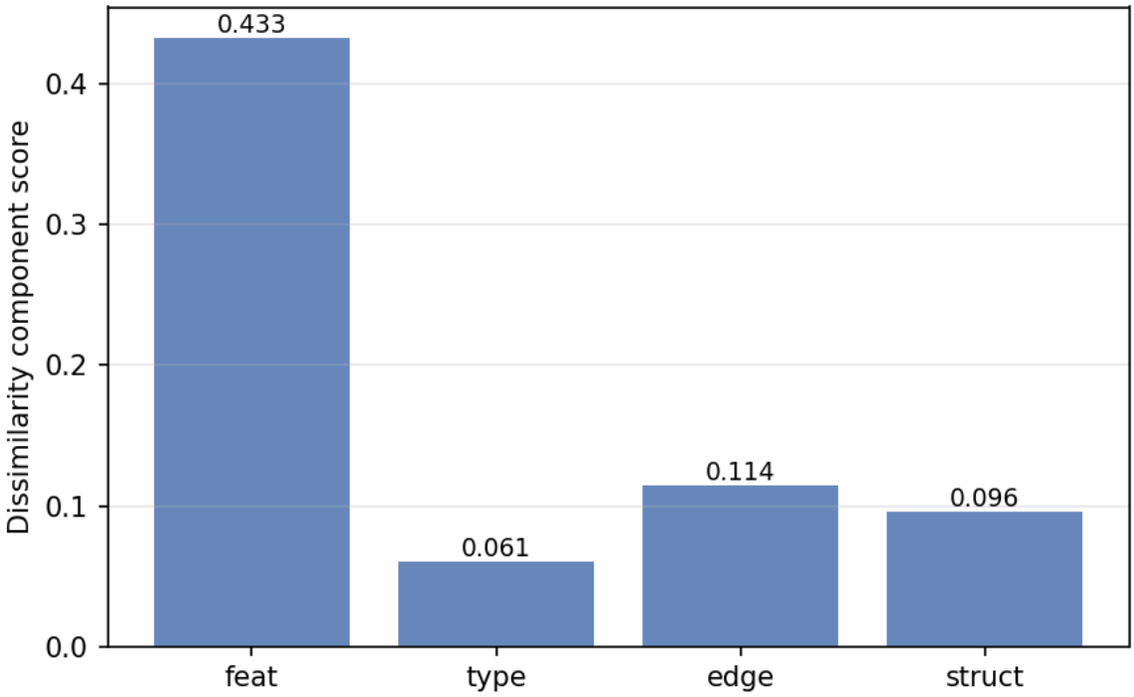


Figure 5.25: Graph representation of the dissimilarity values for the distinct elements compared between the user's chosen graph and the chosen counterfactual.

Rank	Order ID	Predicted (h)	feat	type	edge	struct	Total
1	**1826**	**31.4**	**0.433**	**0.061**	**0.114**	**0.096**	**0.704**
2	1966	65.9	0.499	0.118	0.167	0.010	0.793
3	1822	37.3	0.515	0.184	0.107	0.037	0.844

Figure 5.26: Table showing the other possible counterfactual graph candidates found in the candidate set with their prediction value and dissimilarity scores

Feature	Query (#1801)	CF (#1826)	Δ
price	0.410	-1.208	**-1.618**
n_products	0.083	0.358	**+0.276**
total_weight	-0.701	-0.500	**+0.201**
n_items	-1.206	-1.206	**+0.000**
n_packages	1.719	1.719	**+0.000**

Figure 5.27: Table showing the differences in the features of the Orders node of the two subgraphs. The total delta of the values is shown in the final column.

Activity	Query (#1801)	CF (#1826)	Δ	LOO one-hot Δpred (h)
FailedDelivery	2	1	**-1**	-36.46
ItemOutOfStock	0	1	**+1**	—
PackageDelivered	2	1	**-1**	-1.89
ReorderItem	0	1	**+1**	—

Figure 5.28: Table showing the differences in the features of the Event nodes of the two subgraphs. The number of events of each type is shown for the designated graph along with an estimation of their impact on the model's prediction.

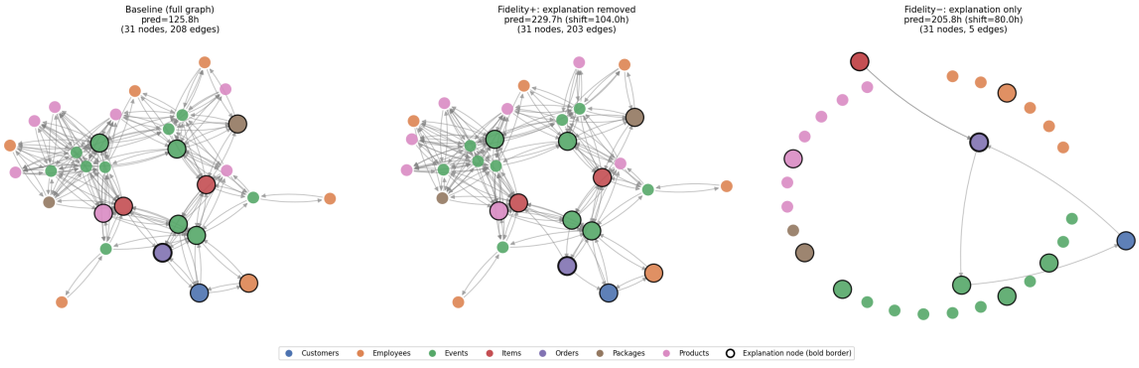


Figure 5.29: Graphs showing the baseline (full graph), fidelity+ (explanation removed), and fidelity- (explanation only) subgraphs used to compute the fidelity metrics meant for the stakeholder.

Metric	Formula	Value	Interpretation
Fidelity+	$\lvert\hat{y} - \Phi(p^{-m}, X^{-m})\rvert$	103.97h	Higher is better (removing the explanation should shift the prediction)
Fidelity−	$\lvert\hat{y} - \Phi(p^{m}, X^{m})\rvert$	80.00h	Closer to 0 is better (explanation alone should reproduce the prediction)
Characterization	Fidelity+ / (Fidelity+ + Fidelity−)	0.565	Higher is better, bounded [0, 1]
Node sparsity	$1 - \lvert m_N\rvert / \lvert N\rvert$	64.5%	Share of the graph excluded from the explanation
Edge sparsity	$1 - \lvert m_A\rvert / \lvert A\rvert$	97.6%	Share of edges excluded from the explanation

Figure 5.30: Table of the explanation metrics obtained.

6 Conclusion

This thesis aimed to address the problem of explainability applied to predictive process monitoring on object-centric event logs. As predictive process monitoring methods become more and more widespread, the reliance of these methods on black-box predictive models such as neural networks may be a detriment for user adoption since research has shown that process stakeholders will only adopt methodologies they feel they can trust. An explainability layer on top of these models can help create trust by providing insight into the elements driving a model's predictions. My proposal is a four-component explainability framework meant to provide stakeholders with clear and understandable insights into the predictive model being used. By providing both factual and counterfactual explanations, the stakeholder is able to understand which specific elements in their inputs are driving the model's prediction as well as how changing different elements in the input can lead to other desired outputs.

In order to test my framework I use two different synthetic OCEL 2.0 datasets meant to emulate the processes of an Order Management and a Logistics company. For each dataset one heterogeneous graph neural network was trained per KPI, and two KPIs were chosen per dataset. The results show that, across the entire prefix set, the heterogeneous architecture is roughly on par with a homogeneous graph-based predictor, with overlapping confidence intervals across all four KPIs and no consistent winner. However, when compared only on last-event cases (which contain the greatest possible amount of information regarding a process execution) the picture becomes clearer. In these cases the heterogeneous model significantly outperformed every baseline on two of the four KPIs (Logistics' Depart and LoadToVehicle), remained the strongest choice but was statistically indistinguishable from the k-dim GNN baseline on a third (Order Management's PayOrder), and underperformed on the fourth (Order Management's PackageDelivered). This shows that the heterogeneous architecture is able to take advantage of richer input information as a process extends, though the benefit is KPI-dependent rather than universal.

My proposed explainability framework was then applied to all four KPI-specific regression models. The experimental evaluation showed that LOO analysis could be used as an efficient way of identifying impactful elements within the input graph, helping to curtail the number of elements for which the computationally costly Shapley Values analysis could be performed on. Similarly, gradient-based methods can be used to calculate the attribution

value of element features in the graph and identify the most impactful elements on the model's prediction. Using these two identifier methods and applying Shapley Values to their chosen elements helped to provide a comprehensive set of input elements and the magnitude of their impact on the model's prediction. These results are then presented to the stakeholder in easy to understand graphs and tables that provide a clear look at the important elements, their rankings and their impact on the given prediction. In order to further create trust in the stakeholder, assessment metrics are also provided for the given factual explanations. Characterization and sparsity are shown to help the stakeholder understand how well the given explanation is at identifying the important elements for the model and separating them from the less important features.

Aside from giving insights into the particular elements driving the prediction, my framework is also able to provide the stakeholder with a counterfactual explanation method to help identify changes in the process that can lead to a desired change such as reducing the time remaining until an event occurs. By presenting these elements in a visually concise way, the stakeholder is able to identify key features in the process that can lead to reductions in the time it takes for a process to finish, or elements that are hindering the process as a whole.

Current limitations for the explainability framework pertain to the limited amount of explainability options available for use with heterogeneous models. While methods like HGExplainer and HTGExplainer exist, they could not be applied in this thesis given certain limitations (specifically for link-prediction/recommender-systems tasks and a failed port from DGL into PyTorch respectively). Additionally, while Shapley Values offer a comprehensive analysis of the features in a graph input and their impact on the prediction it also has a high computational cost which limits its possible applications as process executions grow in size and complexity. A further limitation observed during evaluation concerns the sensitivity of small temporal train/validation/test splits to whichever particular slice of a process's timeline each split happens to cover. For example, for one KPI (Order Management's "Orders to PayOrder"), the validation split's target values were found to be intrinsically less variable than the training split's, which on its own is enough to produce a pattern that could otherwise be mistaken for overfitting. Future work using a validation scheme less sensitive to which particular window of time it draws from, such as rolling-origin or k-fold temporal validation rather than a single fixed split, could mitigate this.

Future work in this space could seek to apply a similar explanation to a global, cross-trace view of the process rather than the current local-only implementation, following the work of de Leoni & Volpato [5]. Similarly, a method seeking to use HTGExplainer could be constructed following a DGL-pipeline in order to validate its use on predictive process monitoring [32].

Ultimately, my proposed framework is able to provide insights into the predictive process monitoring model being applied to object-centric event logs. By providing stakeholders with explanations quantifying the impact of the most important elements of the inputs, metrics that assess the effectiveness of those explanations, and a counterfactual method to compare two distinct process executions, the framework provides a comprehensive look at the driving forces behind the predictive model.

References

[1] IEEE Task Force on Process Mining, W. M. P. van der Aalst *et al.*, "Process mining manifesto," in *Business Process Management Workshops – BPM 2011 International Workshops*, ser. Lecture Notes in Business Information Processing, vol. 99. Springer, 2012, pp. 169–194.

[2] R. Galanti, M. de Leoni, N. Navarin, and A. Marazzi, "Object-centric process predictive analytics," *Expert Systems with Applications*, vol. 213, p. 119173, 2023.

[3] J. N. Adams, G. Park, S. Levich, D. Schuster, and W. M. P. van der Aalst, "A framework for extracting and encoding features from object-centric event data," 2022.

[4] T. K. Smit, H. A. Reijers, and X. Lu, "HOEG: A new approach for object-centric predictive process monitoring," 2024.

[5] M. de Leoni and P. Volpato, "Global predictive monitoring of object-centric processes," in *Business Process Management*, A. Senderovich, C. Cabanillas, I. Vanderfeesten, and H. A. Reijers, Eds. Cham: Springer Nature Switzerland, 2026, pp. 255–272.

[6] R. Galanti and M. de Leoni, "Predictive analytics for object-centric processes: Do graph neural networks really help?" in *Business Process Management Workshops*, J. De Weerdt and L. Pufahl, Eds. Cham: Springer Nature Switzerland, 2024, pp. 521–533.

[7] J. Li, W. Monroe, and D. Jurafsky, "Understanding neural networks through representation erasure," 2017.

[8] J. Castro, D. Gómez, and J. Tejada, "Polynomial calculation of the Shapley value based on sampling," *Computers & Operations Research*, vol. 36, pp. 1726–1730, 2009.

[9] M. Sundararajan, A. Taly, and Q. Yan, "Axiomatic attribution for deep networks," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017, integrated Gradients.

[10] T.-H. Huang, A. Metzger, and K. Pohl, "Counterfactual explanations for predictive business process monitoring," 2022, lORELEY.

[11] A. Stevens, J. De Smedt, and J. Peeperkorn, "Quantifying explainability in outcome-oriented predictive process monitoring," in *Process Mining Workshops – ICPM 2021 International Workshops*, ser. Lecture Notes in Business Information Processing, vol. 433. Springer, 2022, kU Leuven.

[12] W. Gherissi, J. El Haddad, and D. Grigori, "Object centric predictive process monitoring," in *Service-Oriented Computing – ICSOC 2022 Workshops*, ser. Lecture Notes in Computer Science, vol. 13821. Springer, 2023, pp. 27–39.

[13] M. de Leoni, W. M. P. van der Aalst, and M. Dees, "A general process mining framework for correlating, predicting and clustering dynamic behavior based on event logs," *Information Systems*, vol. 56, pp. 235–257, 2016, no local copy available; read online.

[14] B. Khemani, S. Patil, K. Kotecha, and S. Tanwar, "A review of graph neural networks: concepts, architectures, techniques, challenges, datasets, applications, and future directions," *Journal of Big Data*, vol. 11, no. 1, p. 18, 2024.

[15] J. N. Adams, G. Park, and W. M. P. van der Aalst, "Preserving complex object-centric graph structures to improve machine learning tasks in process mining," *Engineering Applications of Artificial Intelligence*, vol. 125, p. 106764, 2023, no local copy available; read online.

[16] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 4765–4774.

[17] R. Galanti, M. de Leoni, M. Monaro, N. Navarin, A. Marazzi, B. Di Stasi, and S. Maldera, "An explainable decision support system for predictive process analytics," *Engineering Applications of Artificial Intelligence*, vol. 120, p. 105904, 2023.

[18] N. Mehdiyev, M. Majlatow, and P. Fettke, "Interpretable and explainable machine learning methods for predictive process monitoring: a systematic literature review," *Artificial Intelligence Review*, vol. 58, no. 378, 2025.

[19] S. Weinzierl, S. Zilker, J. Brunk, K. Revoredo, M. Matzner, and J. Becker, "Xnap: Making lstm-based next activity predictions explainable by using lrp," in *Business Process Management Workshops*, A. Del Río Ortega, H. Leopold, and F. M. Santoro, Eds. Cham: Springer International Publishing, 2020, pp. 129–141.

[20] B. Wickramanayake, Z. He, C. Ouyang, C. Moreira, Y. Xu, and R. Sindhgatta, "Building interpretable models for business process prediction using shared and specialised attention mechanisms," *Knowledge-Based Systems*, vol. 248, p. 108773, July 2022. [Online]. Available: http://dx.doi.org/10.1016/j.knosys.2022.108773

[21] V. Pasquadibisceglie, G. Castellano, A. Appice, and D. Malerba, "Fox: a neuro-fuzzy model for process outcome prediction and explanation," in *2021 3rd International Conference on Process Mining (ICPM)*, 2021, pp. 112–119.

[22] C. Hsieh, C. Moreira, and C. Ouyang, "Dice4el: Interpreting process predictions using a milestone-aware counterfactual approach," 2021. [Online]. Available: https://arxiv.org/abs/2107.08697

[23] W. Gherissi, M. Acheli, J. E. Haddad, and D. Grigori, "Predictive process monitoring using object-centric graph embeddings," in *Service-Oriented Computing – ICSOC 2024 Workshops*, S. Kallel, C. Raibulet, I. Bouassida Rodriguez, N. Faci, A. Bennaceur, S. Cheikhrouhou, L. Ben Ayed, M. Sellami, E. Y. Nakagawa, and R. Ben Halima, Eds. Singapore: Springer Nature Singapore, 2026, pp. 55–67.

[24] M. Nandan, S. Mitra, and D. De, "GraphXAI: a survey of graph neural networks (GNNs) for explainable AI (XAI)," *Neural Computing and Applications*, vol. 37, pp. 10 949–11 000, 2025.

[25] R. Ying, D. Bourgeois, J. You, M. Zitnik, and J. Leskovec, "GNNExplainer: Generating explanations for graph neural networks," in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019, pp. 9244–9255.

[26] D. Luo, W. Cheng, D. Xu, W. Yu, B. Zong, H. Chen, and X. Zhang, "Parameterized explainer for graph neural network," in *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020, pGExplainer.

[27] X. Wang and H.-W. Shen, "GNNInterpreter: A probabilistic generative model-level explanation for graph neural networks," in *Proceedings of the 11th International Conference on Learning Representations (ICLR)*, 2023.

[28] H. Yuan, H. Yu, S. Gui, and S. Ji, "Explainability in graph neural networks: A taxonomic survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 5, pp. 5782–5799, 2022, also available as an arXiv preprint.

[29] M. A. Prado-Romero, B. Prenkaj, G. Stilo, and F. Giannotti, "A survey on graph counterfactual explanations: Definitions, methods, evaluation, and research challenges," *ACM Computing Surveys*, vol. 56, no. 7, p. 171, 2024.

[30] D. Brughmans, P. Leyman, and D. Martens, "NICE: An algorithm for nearest instance counterfactual explanations," 2022.

[31] G. P. Mika, A. Bouzeghoub, K. Węgrzyn Wolska, and Y. M. Neggaz, "HGExplainer: Explainable heterogeneous graph neural network," 2023.

[32] J. Li, C. Zhang, and C. Zhang, "Heterogeneous temporal graph neural network explainer," in *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management (CIKM)*, 2023, hTGExplainer.

[33] X. Zhai, J. Jiang, A. Dejl, A. Rago, F. Guo, F. Toni, and A. Sivakumar, "Heterogeneous graph neural networks with post-hoc explanations for multi-modal and explainable land use inference," *Information Fusion*, vol. 120, p. 103057, 2025.

[34] B. Knopp, "Order management object-centric event log," 2023. [Online]. Available: https://www.ocel-standard.org/event-logs/simulations/order-management/

[35] B. Knopp and N. Graves, "Container logistics object-centric event log," 2023. [Online]. Available: https://www.ocel-standard.org/event-logs/simulations/logistics/

Acknowledgments